

Por qué automatizar la infraestructura

Parte 1 de 4 · Unidad 2

Ing. Rodolfo Cañas Cervantes

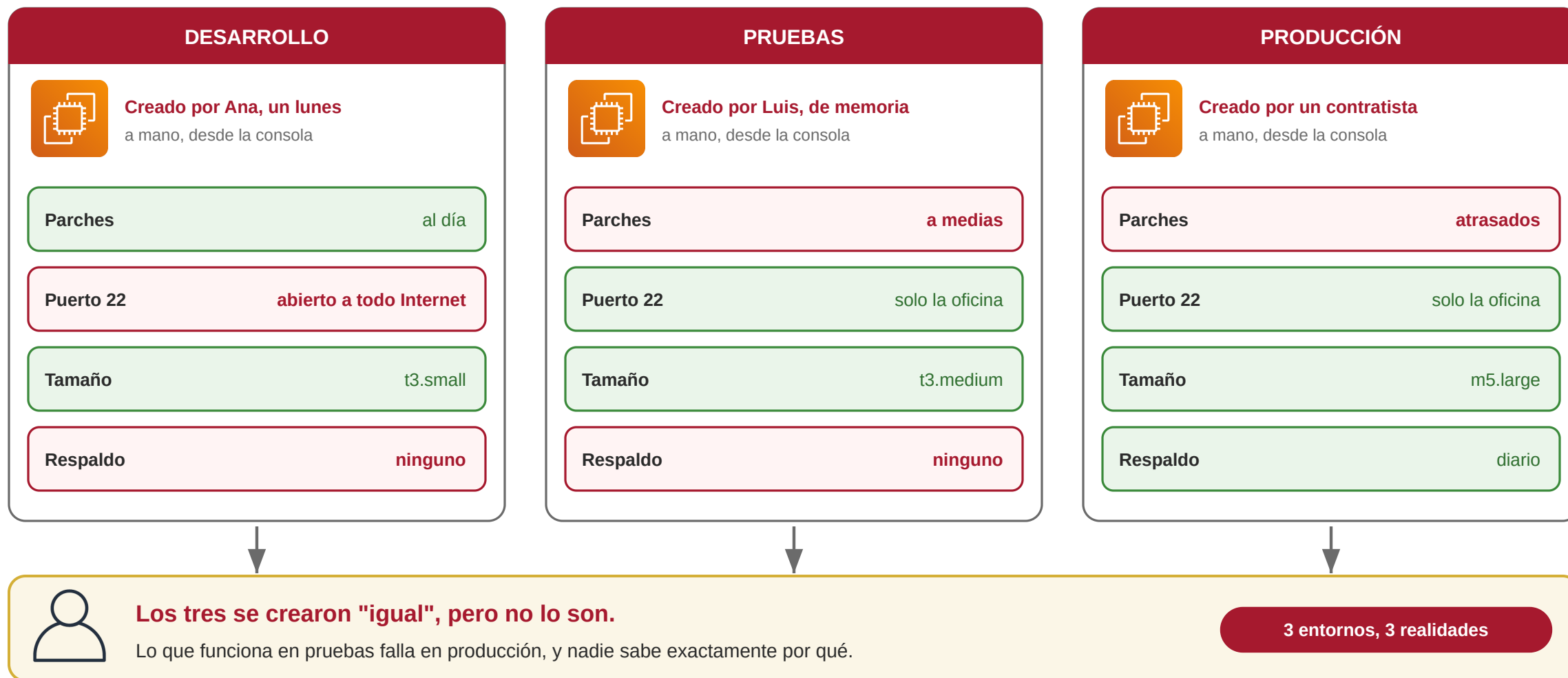
Universidad de la Costa (CUC) · Ingeniería de Sistemas

Semana 11 · Periodo 2026-2 · Barranquilla, Colombia

Consola, scripts e infraestructura como código: qué cambia cuando el plano de la nube se vuelve un archivo.

Tres entornos "iguales" que no lo son

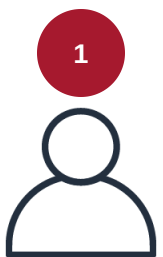
El problema de la infraestructura manual no es el clic: es que nadie puede repetir exactamente lo que hizo.



Cuatro partes para automatizar y repasar

Hoy cierra la Unidad 2: aprendemos a construir con código todo lo que ya sabíamos construir con clics.

Una hora de clase = cuatro videos de unos 15 minutos



Por qué automatizar

Consola, scripts e IaC
Imperativo y declarativo
Idempotencia y deriva

video de 15 min



IaC en la práctica

CloudFormation: pila,
conjuntos de cambios, deriva
Terraform y Bicep

video de 15 min



Entrega continua

Git, pipelines CI/CD
Entornos dev, test, prod
Pruebas y GitOps

video de 15 min



Repaso U2 y lab

Mapa de S07 a S10
Arquitectura de referencia
Lab IaC y Parcial 2

video de 15 min



Lab de automatización (IaC): 10 % de la nota

Se construye con la Parte 2 y la Parte 4.

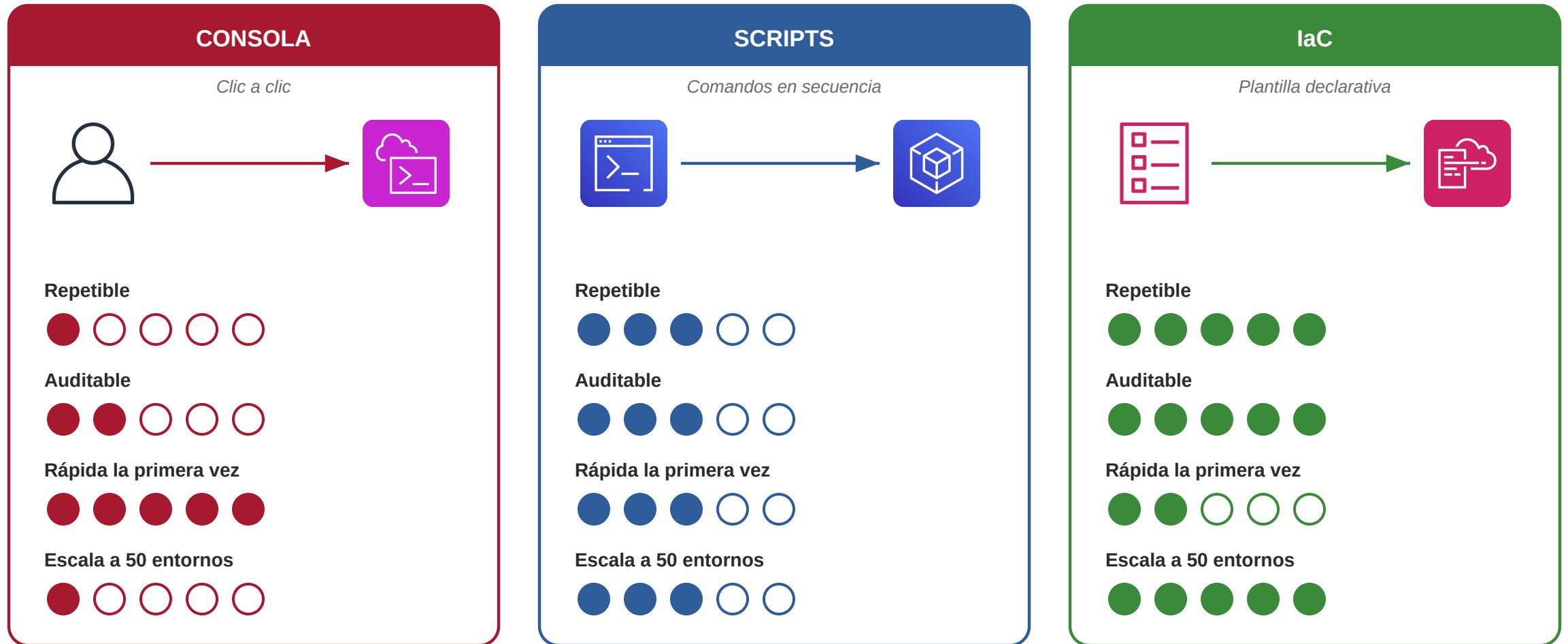


Semana del Parcial 2: 20 % de rúbrica

Cubre las semanas 7 a 11 de la Unidad 2.

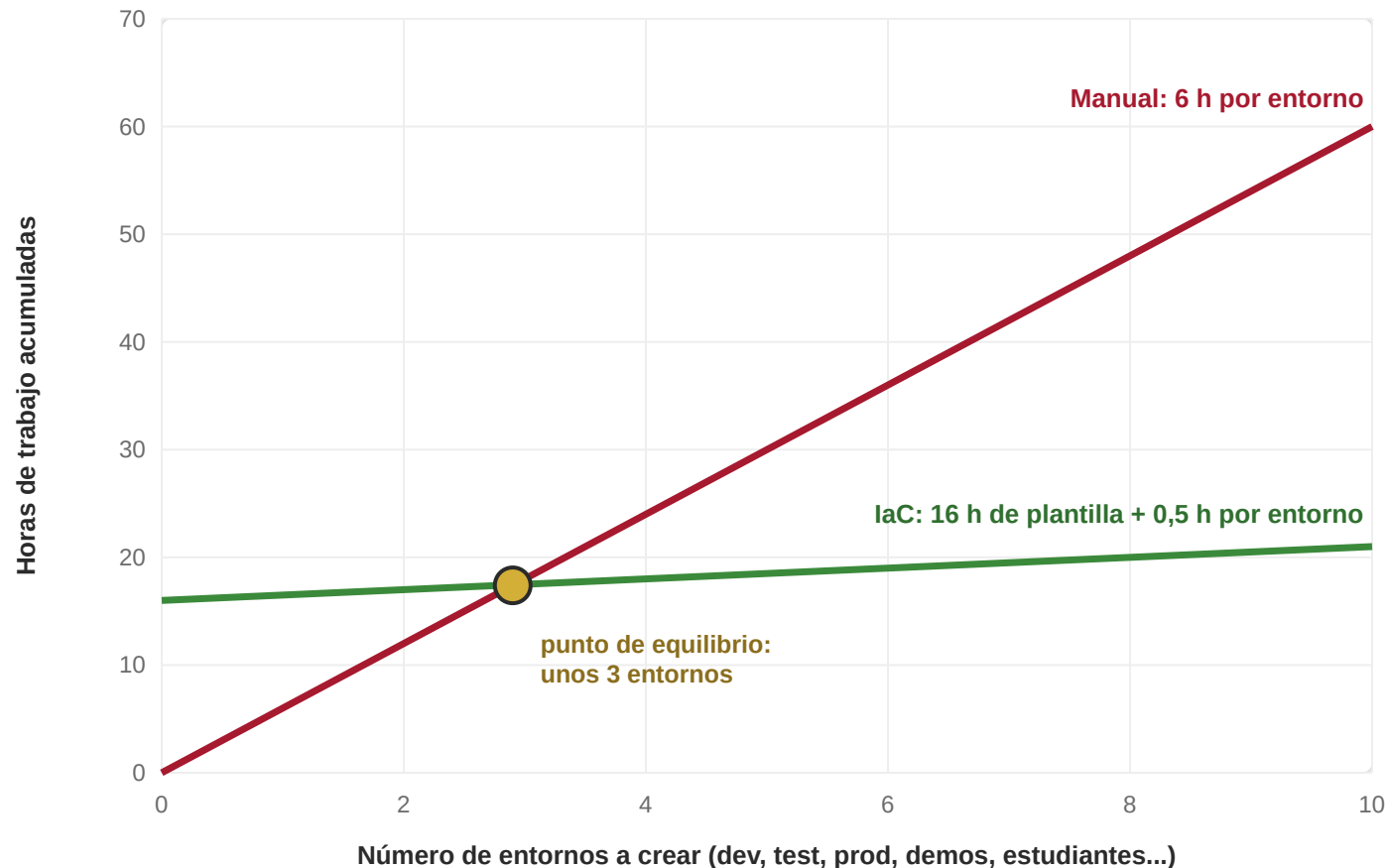
Tres formas de crear infraestructura

La consola gana en la primera vez; el código gana desde la segunda.



El costo oculto del clic

Armado dos entornos a mano parece barato; armar diez revela el costo real.



Cifras ilustrativas para mostrar la forma de las curvas, no mediciones reales.



Cada clic es una oportunidad de error

Un entorno armado a mano rara vez es igual al anterior.



El primer día cuesta más

Escribir la plantilla es una inversión que se paga rápido.

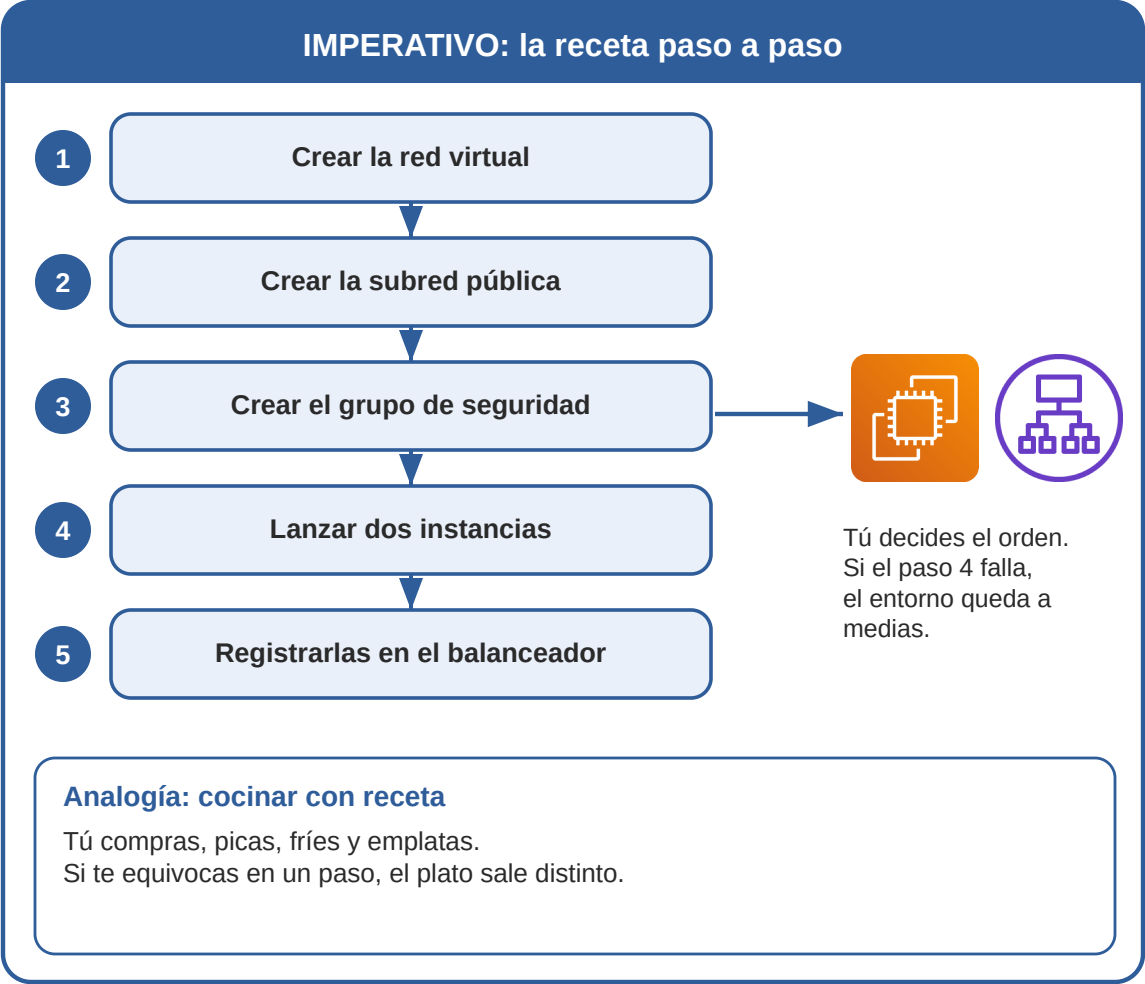


Después, casi gratis

El entorno 10 cuesta lo mismo que el entorno 2: segundos de ejecución.

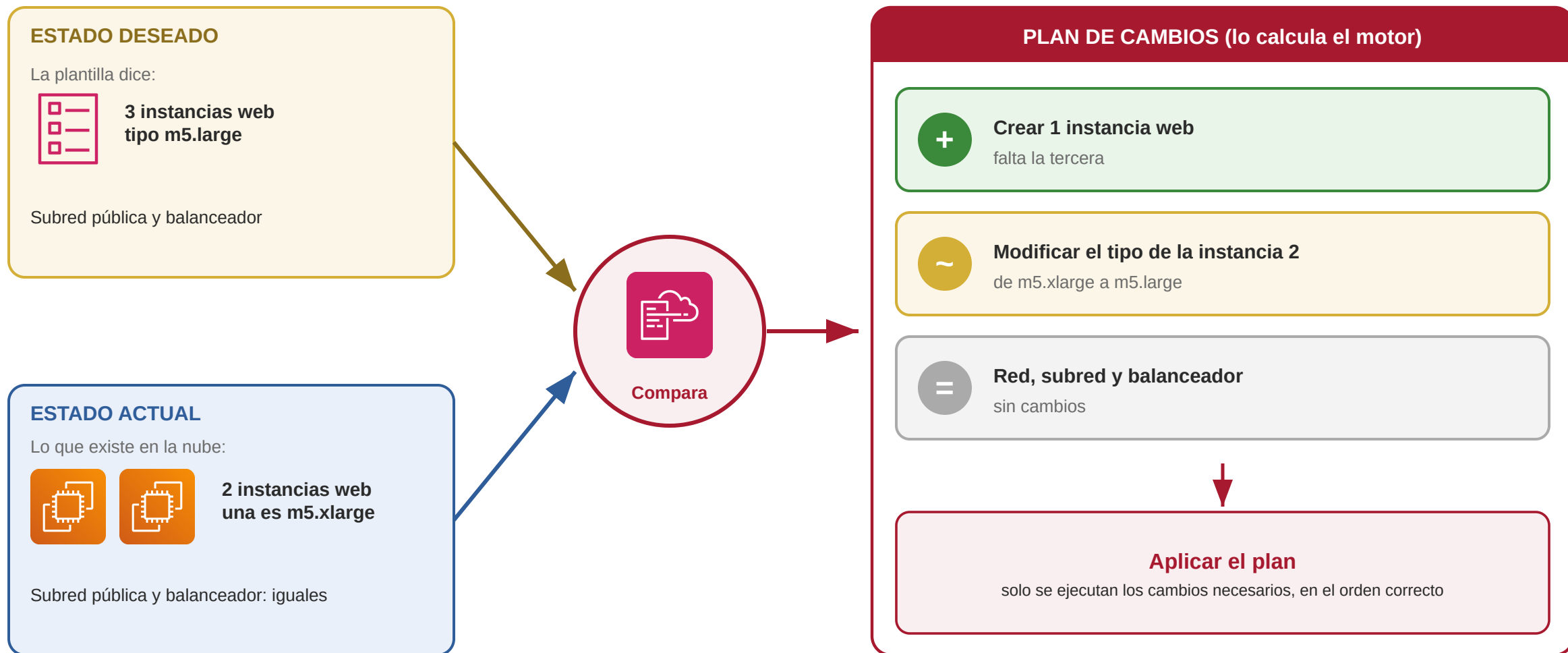
Imperativo frente a declarativo

Imperativo es decir cómo hacerlo; declarativo es decir cómo debe quedar.



Cómo piensa un motor declarativo

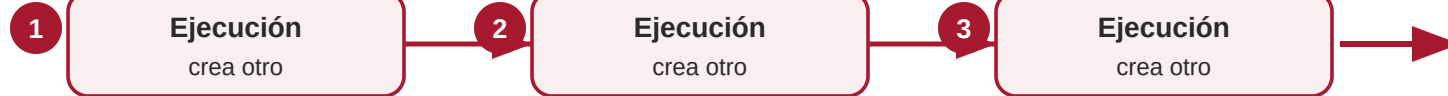
Compara lo que quieres con lo que existe y ejecuta solo la diferencia.



Idempotencia: repetir sin miedo

Una operación idempotente deja el mismo resultado la primera vez y las diez siguientes.

Script ingenuo: "crear un servidor"



Un reintento por falla de red duplica los recursos.

$f(f(x))$ distinto de $f(x)$

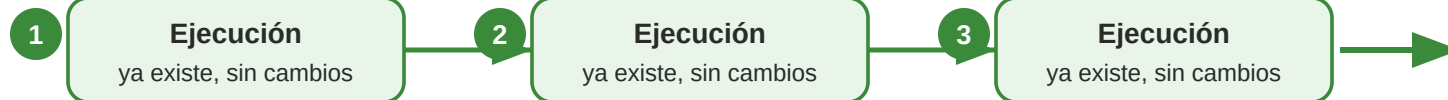
Resultado en la nube



3 servidores, 3 facturas

cada ejecución agrega uno

Declarativo: "debe existir 1 servidor"



Un reintento por falla de red no cambia nada: compara y sigue.

$f(f(x)) = f(x)$

Resultado en la nube



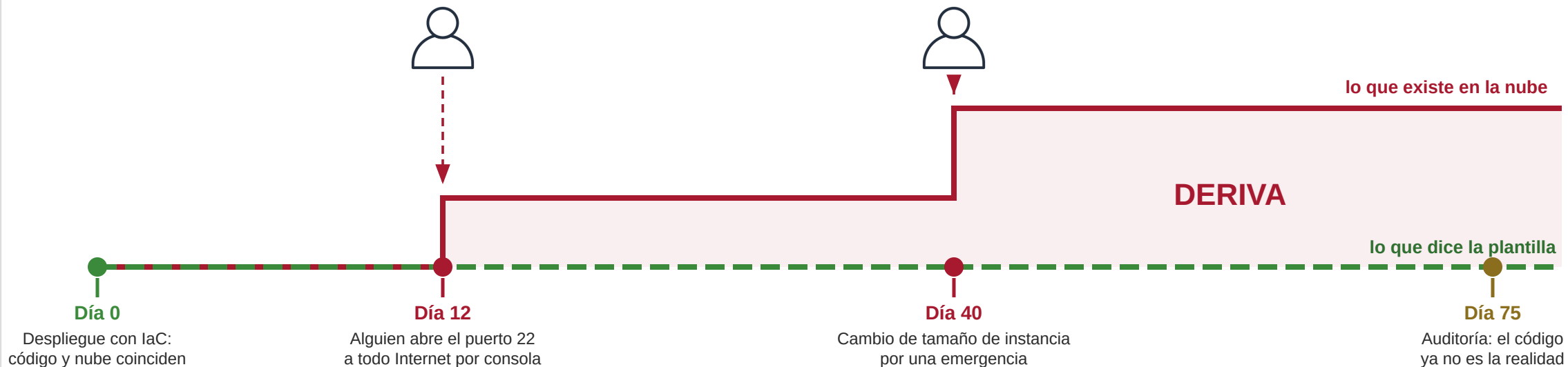
1 servidor, siempre

repetir no cambia el resultado

Deriva de configuración: el código que miente

Cada cambio manual aleja la realidad del código, y la diferencia crece sin que nadie la vea.

Qué dice el código frente a qué hay realmente



Deriva (drift)

Diferencia entre la infraestructura declarada en el código y la que existe de verdad.

Por qué ocurre

Cambios manuales de emergencia, scripts sueltos, otro equipo con permisos amplios.

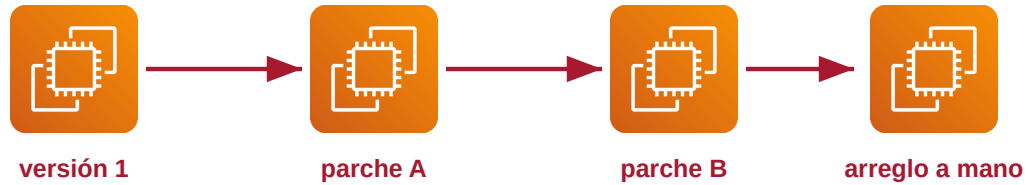
Por qué duele

La próxima ejecución puede borrar el arreglo manual o fallar al chocar con él.

Infraestructura mutable frente a inmutable

En la inmutable no se arregla un servidor: se reemplaza por uno nuevo construido desde una imagen.

MUTABLE: la mascota que se cura

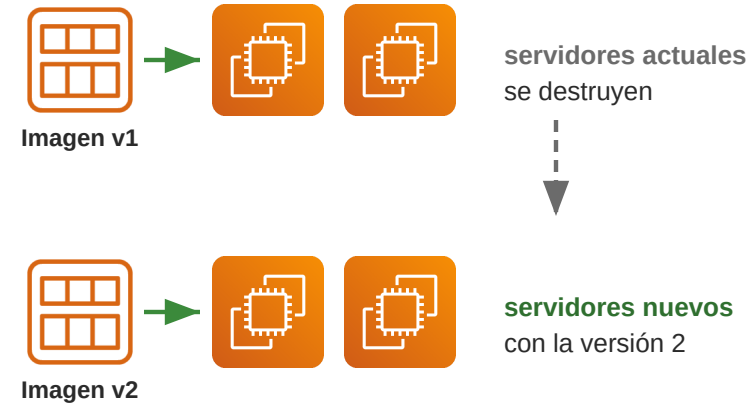


El mismo servidor se modifica en el sitio, una y otra vez.

Consecuencias

- Nadie sabe qué tiene instalado exactamente.
- No se puede reproducir si se daña.
- Cada servidor termina siendo un "copo de nieve" único.
- Para entenderlo hay que entrar a revisarlo.

INMUTABLE: el ganado que se reemplaza

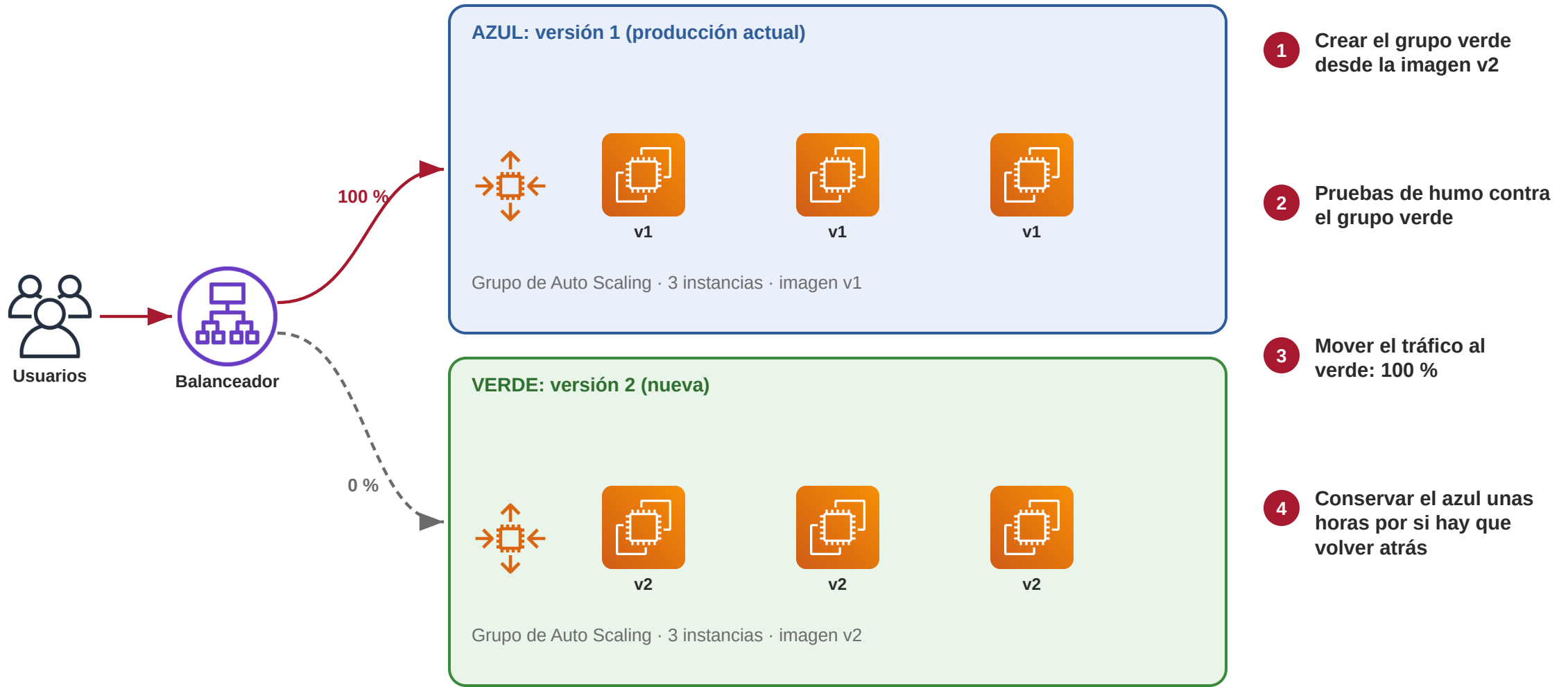


Consecuencias

- Cada servidor sale de una imagen conocida.
- Cambiar es reemplazar, no tocar.
- Si algo falla, se vuelve a la imagen anterior.
- Nadie entra por SSH a "arreglar".

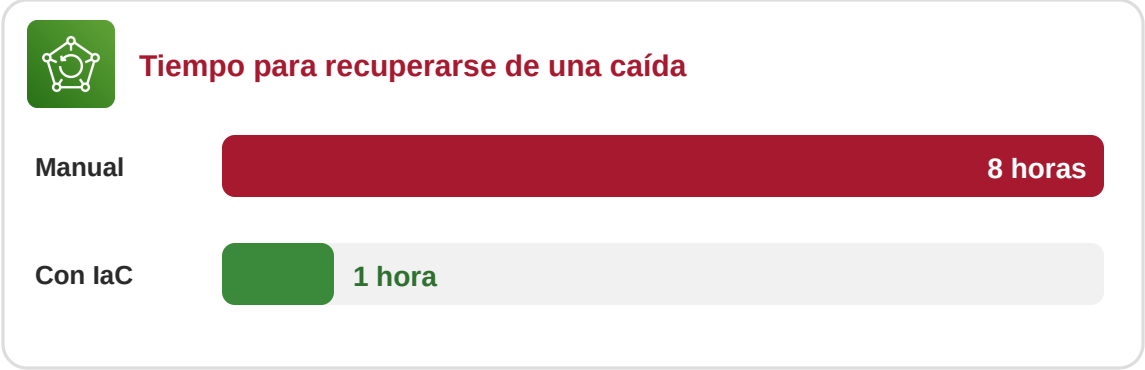
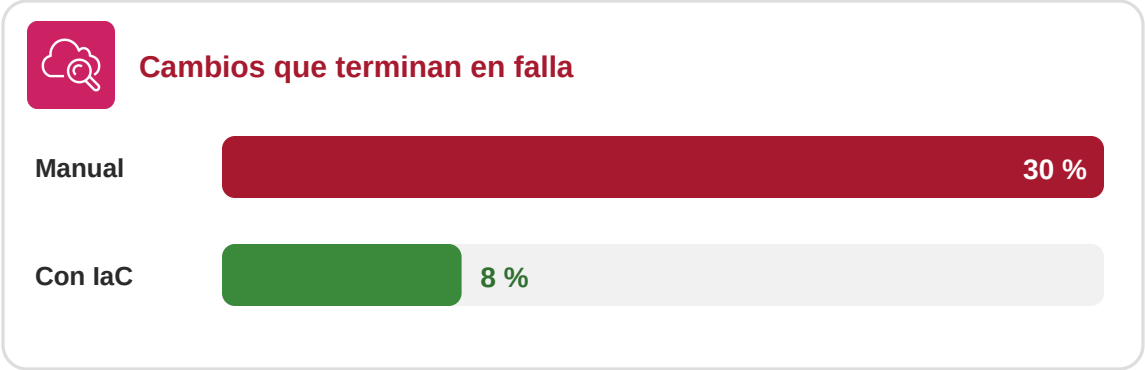
Reemplazo azul-verde: inmutable en acción

Dos entornos completos conviven; el tráfico cambia de uno al otro y la vuelta atrás es instantánea.



Beneficios que se pueden medir

Automatizar no es una moda: mejora tiempos, fallos y recuperación de forma verificable.



Las cuatro métricas de DORA que mejoran con automatización:

Frecuencia de despliegue

Tiempo de entrega de un cambio

Tasa de fallos de cambios

Tiempo de recuperación

Valores ilustrativos del caso CIC

IaC en los pilares de Well-Architected

Un solo hábito, escribir la infraestructura como código, mejora los seis pilares del marco.

Infraestructura como código



Excelencia operativa

Operaciones como código: todo cambio es versionado y repetible

principio directo



Seguridad

Cambios revisados y auditable; sin accesos manuales



Fiabilidad

Reconstruir un entorno completo en minutos



Eficiencia de rendimiento

Probar otros tamaños con solo cambiar un parámetro



Optimización de costos

Crear y borrar entornos bajo demanda



Sostenibilidad

Solo lo necesario, apagado automático fuera de horario

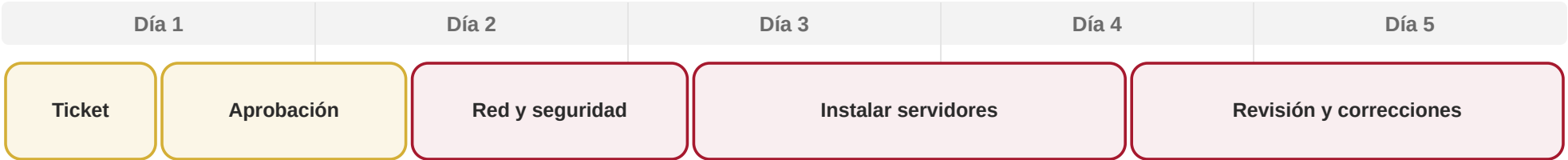
Caso CIC: antes y después de automatizar

El mismo pedido de infraestructura pasa de unos 5 días a unos 25 minutos.



Corporación Industrial del Caribe, Barranquilla: necesita un entorno para su nuevo sistema de pedidos de planta.

ANTES
manual



Cada persona lo hace distinto y queda en su memoria.



DESPUÉS
con código



La plantilla es la documentación viva

Cualquiera la lee, la revisa y la reutiliza para los entornos de pruebas y capacitación. Nada depende de la memoria de una persona.

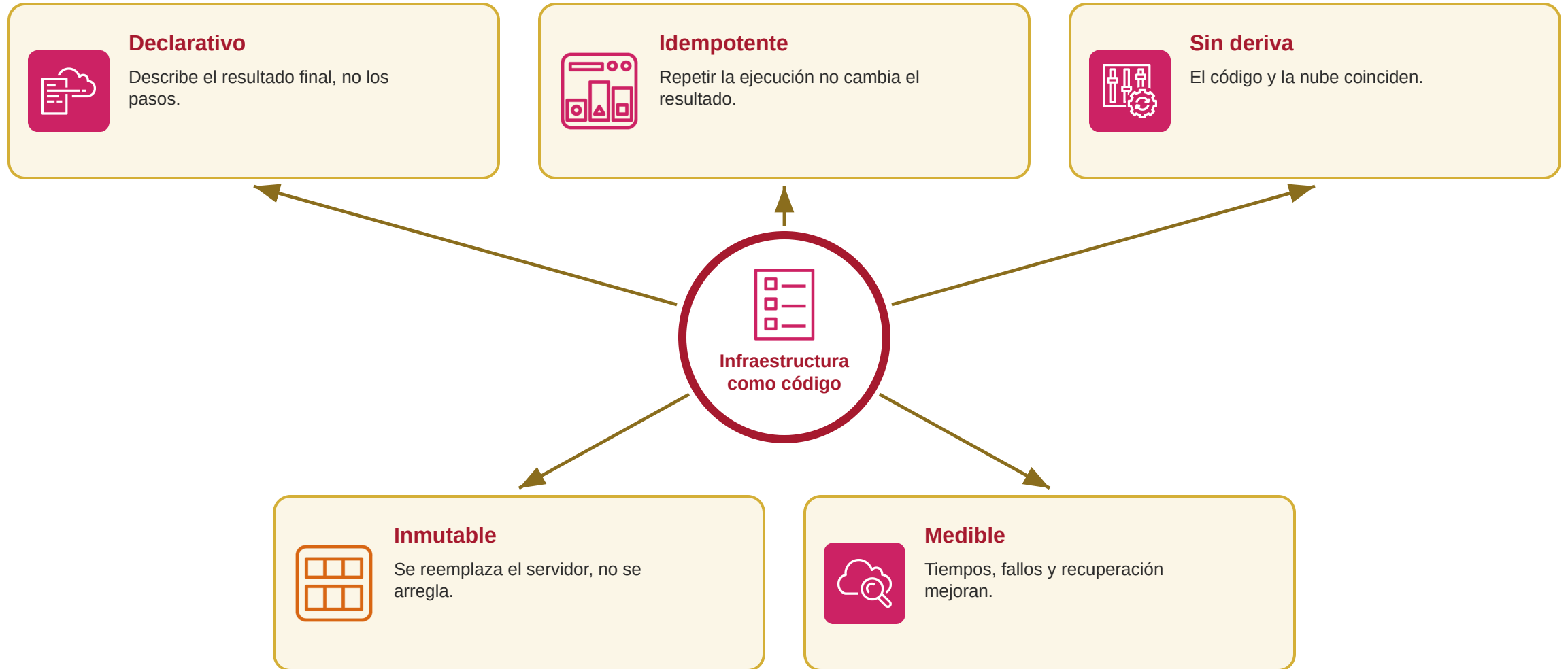
Lección

El tiempo no se ahorra en el despliegue: se ahorra en no tener que volver a pensarlo cada vez.

Cifras ilustrativas.

Lo que queda de la Parte 1

Cinco ideas para llevar: declarar, repetir sin miedo, evitar la deriva, reemplazar y medir.



Infraestructura como código en la práctica

Parte 2 de 4 · Unidad 2

Ing. Rodolfo Cañas Cervantes

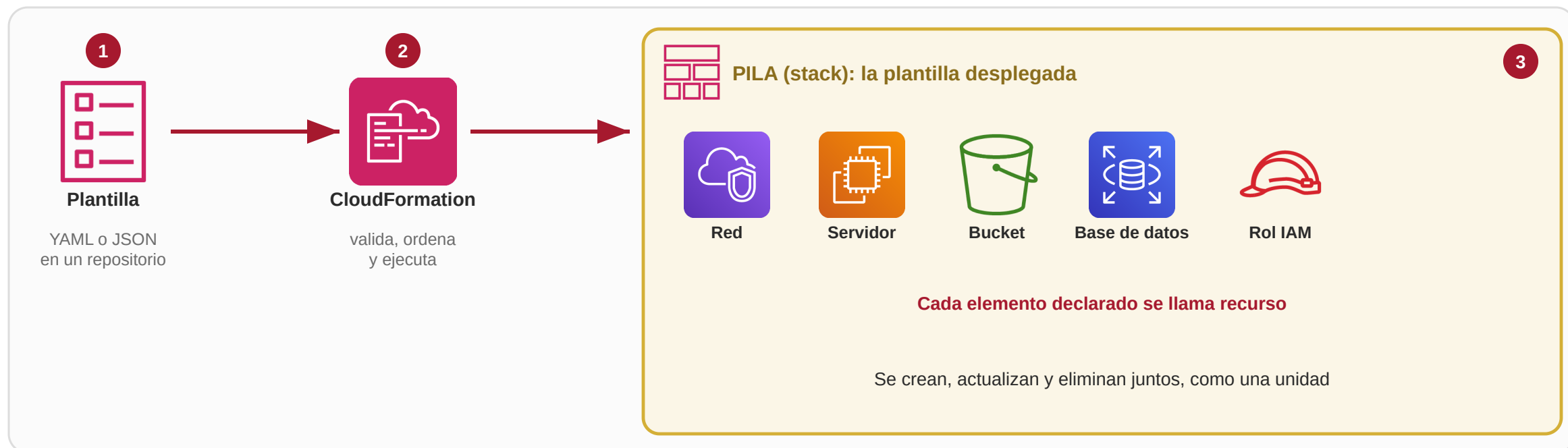
Universidad de la Costa (CUC) · Ingeniería de Sistemas

Semana 11 · Periodo 2026-2 · Barranquilla, Colombia

CloudFormation de punta a punta, y Terraform y Bicep como alternativas multi-nube.

La plantilla es el plano; la pila, la obra

CloudFormation convierte un archivo de texto en recursos reales y los administra como una sola unidad.



Plantilla

El plano: describe qué debe existir.



Pila

Una instancia de ese plano en una cuenta y región.

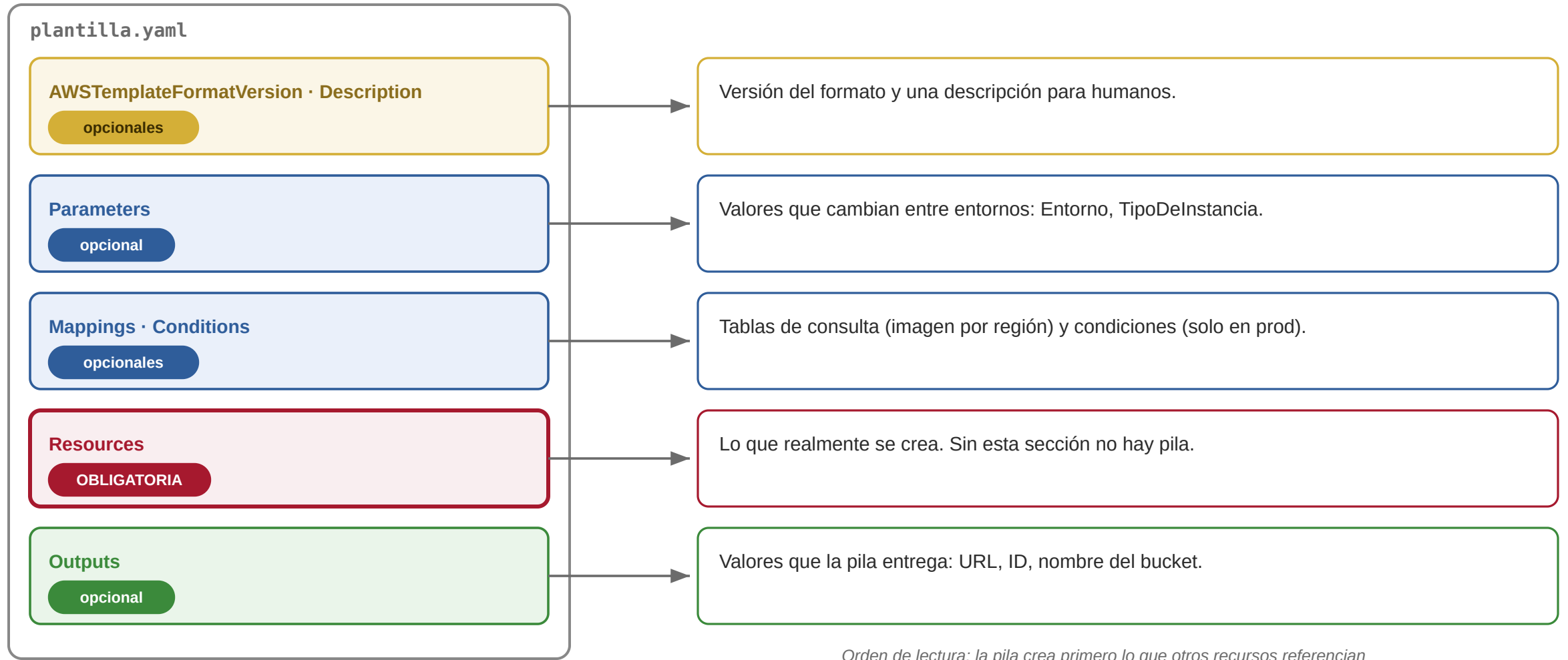


Recurso

Cada pieza concreta: red, servidor, bucket.

Anatomía de una plantilla CloudFormation

Solo Resources es obligatoria; el resto hace la plantilla reutilizable y legible.



Una plantilla corta, leída como diagrama

Parámetro de entrada, recurso en el centro y salida: tres bloques que se repiten en toda plantilla.

```
reportes.yaml
AWSTemplateFormatVersion: "2010-09-09"
Parameters:
  Entorno:
    Type: String
    AllowedValues: [dev, test, prod]
Resources:
  Bucket:
    Type: AWS::S3::Bucket
    Properties:
      BucketName: !Sub cic-reportes-${Entorno}-${AWS::AccountId}
      VersioningConfiguration:
        Status: Enabled
Outputs:
  NombreBucket:
    Value: !Ref Bucket
```

PARÁMETRO: Entorno

Al crear la pila eliges dev, test o prod. El mismo archivo sirve para los tres entornos.

RECURSO: Bucket



Un bucket de S3 con versionado activado. El nombre se arma con !Sub: entorno + número de cuenta.

SALIDA: NombreBucket

!Ref Bucket devuelve el nombre real; otras pilas pueden importarlo.

Quince líneas bastan para crear un recurso real, repetible y revisable.

Al crear la pila con Entorno = dev



cic-reportes-dev-111122223333

versionado: Enabled · cuenta de ejemplo

Parámetros y funciones intrínsecas

Con !Ref y !GetAtt un recurso apunta a otro, y CloudFormation deduce el orden de creación.

Funciones intrínsecas

!Ref

Devuelve el valor de un parámetro o el ID de un recurso

!Sub

Sustituye variables dentro de un texto

!GetAtt

Lee un atributo del recurso, como su IP o su ARN

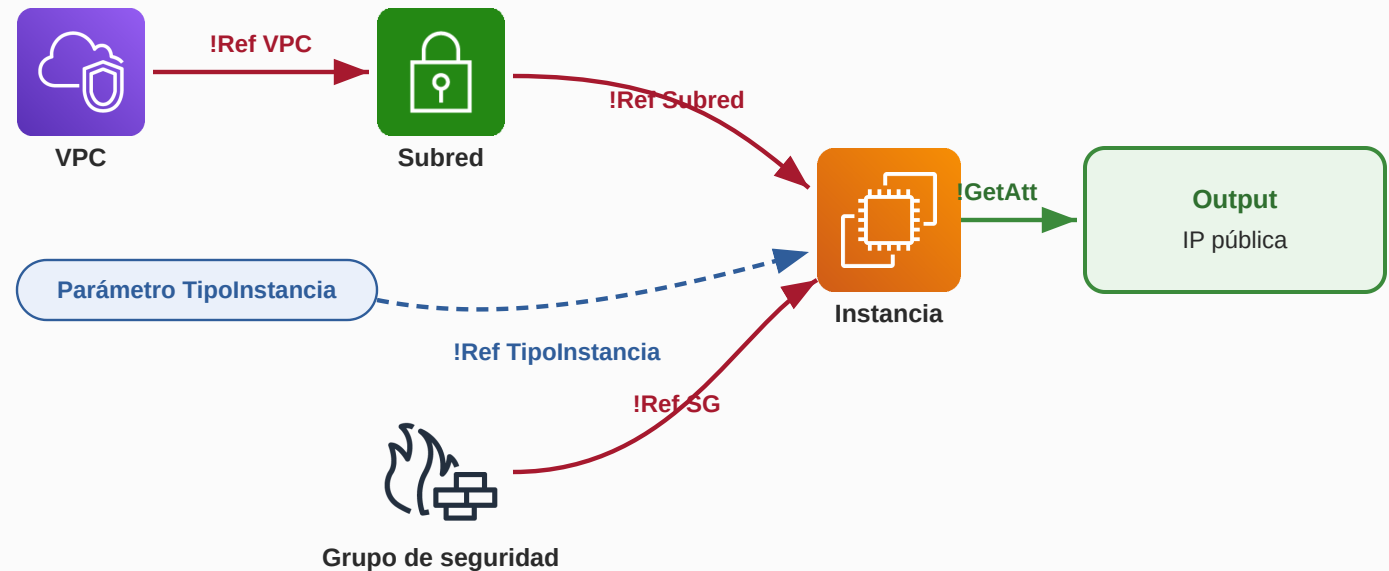
!FindInMap

Busca un valor en una tabla de Mappings

!If

Elige un valor según una condición

Dependencias: CloudFormation las deduce de las referencias



Orden de creación resultante

VPC

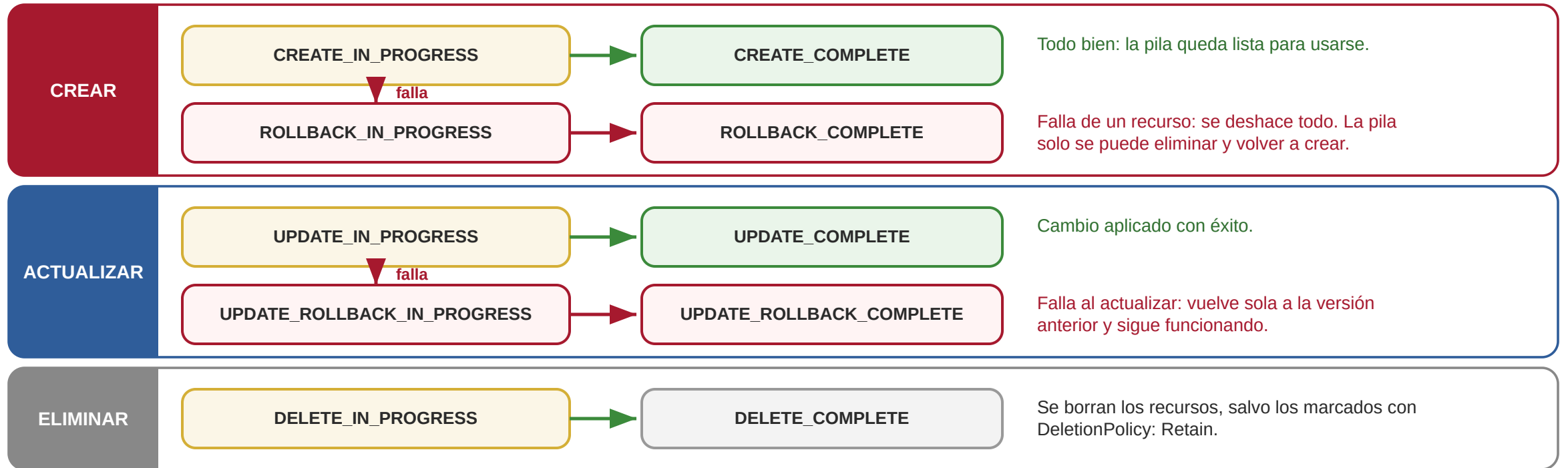
Subred y SG

Instancia

Output

Ciclo de vida de una pila

Cada operación termina en un estado final claro; un fallo no deja recursos a medias.



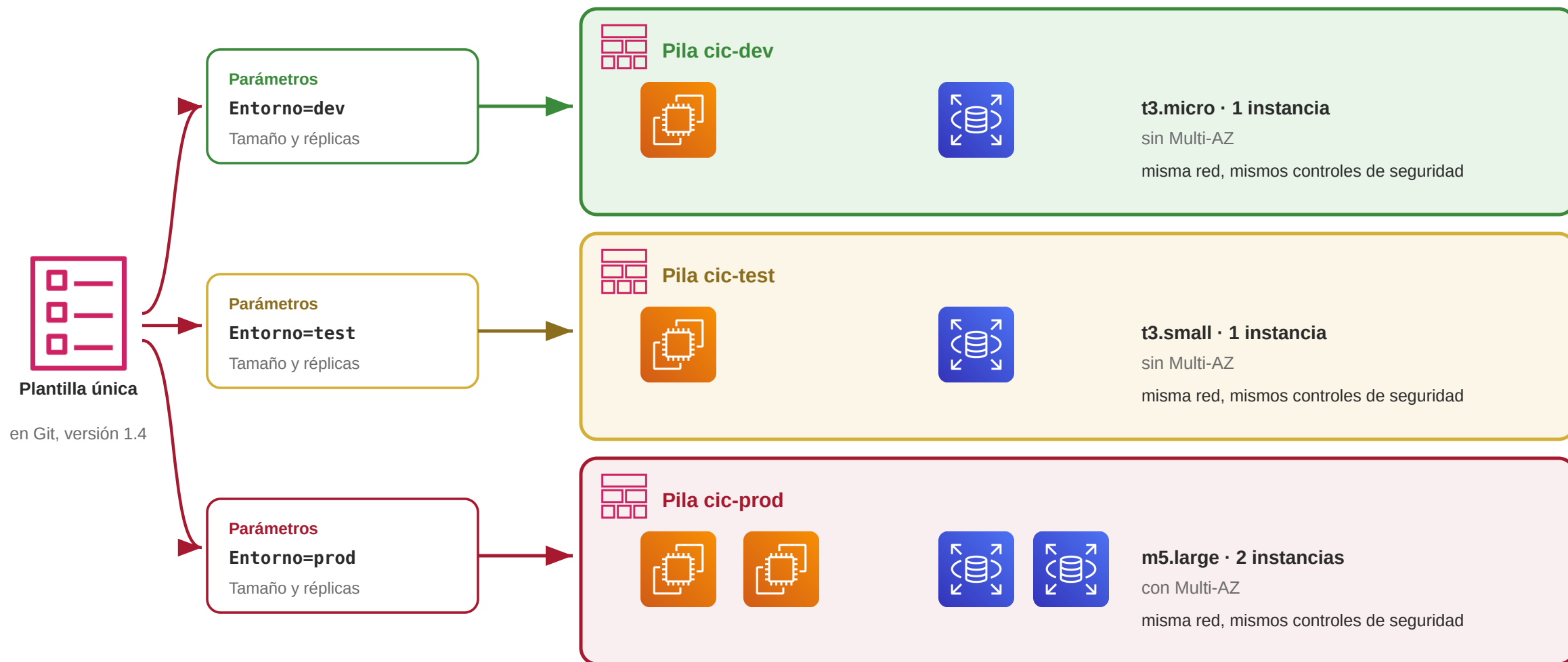
Dónde mirar cuando algo falla

Consola de CloudFormation, pestaña Eventos: el primer evento FAILED indica la causa raíz.

Los eventos posteriores suelen ser consecuencia, no causa.

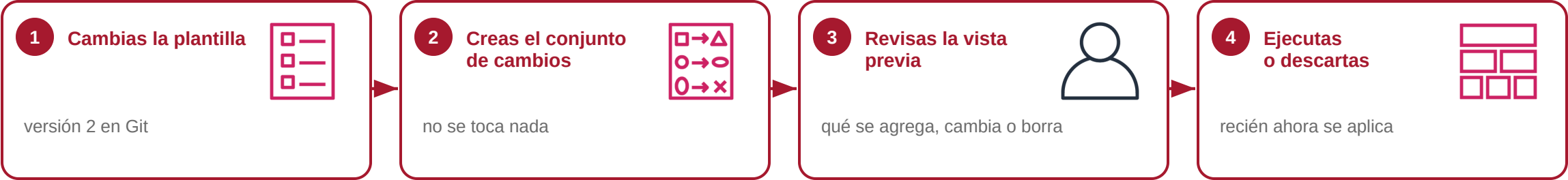
Un mismo plano, tres entornos

Los parámetros cambian el tamaño; la forma y los controles son idénticos en dev, test y prod.



Conjuntos de cambios: ver antes de aplicar

Un conjunto de cambios muestra qué se agrega, modifica o reemplaza sin tocar la pila.



Vista previa del conjunto de cambios (ejemplo)

Acción	ID lógico	Tipo de recurso	Reemplazo
Modify	Instancia	AWS::EC2::Instance	True Cambió ImageId
Modify	GrupoSeguridad	AWS::EC2::SecurityGroup	False Regla modificada
Add	AlarmaCPU	AWS::CloudWatch::Alarm	— Recurso nuevo
Remove	BucketTemporal	AWS::S3::Bucket	— Ya no está

Atención al reemplazo

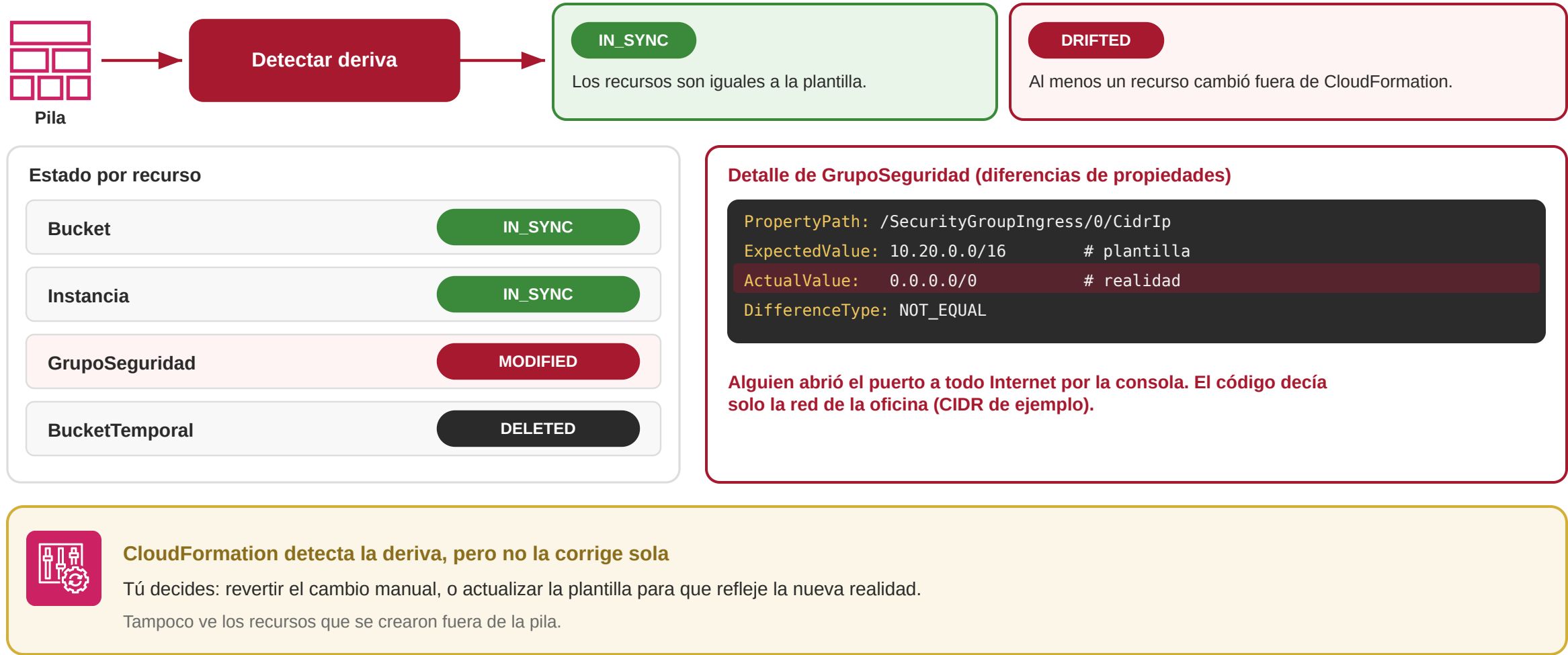
Replacement True: se destruye el recurso y se crea otro nuevo. Si guardaba datos, se pierden.

Por qué importa

Es el equivalente a un "plan": ves el efecto antes de pagar sus consecuencias.

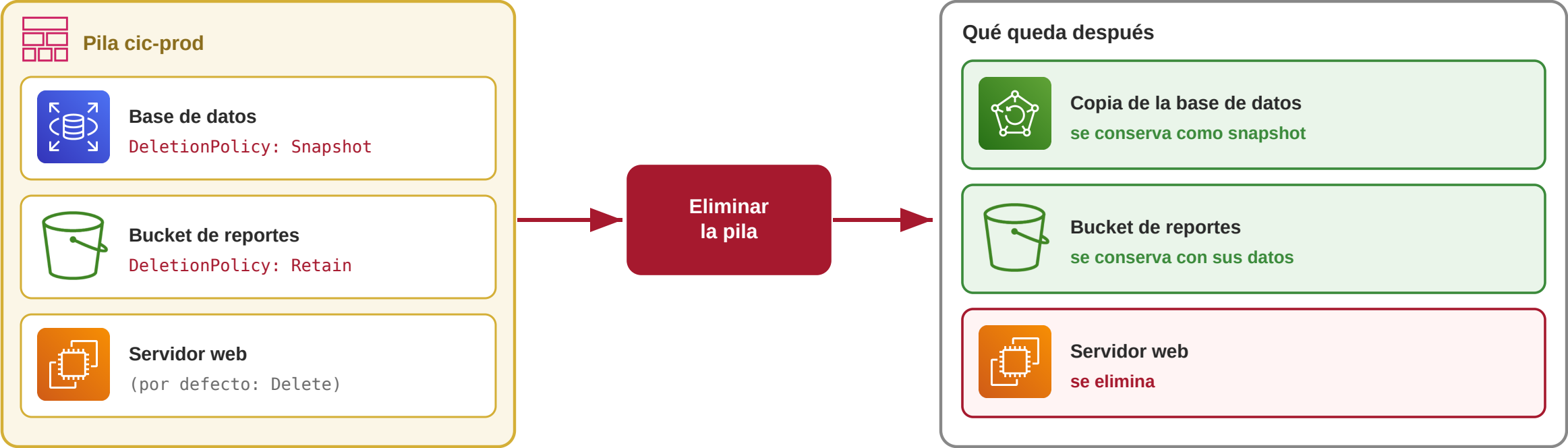
Detección de deriva en CloudFormation

Compara lo que dice la plantilla con lo que existe y marca los recursos que cambiaron por fuera.



Proteger lo que no se puede perder

DeletionPolicy y protecciones de pila evitan que borrar un entorno destruya datos valiosos.



Tres capas de protección contra borrados accidentales



Terraform: init, plan y apply

Un solo flujo y un lenguaje, HCL, para administrar recursos en muchas nubes.

main.tf (HCL)

```
resource "aws_s3_bucket" "reportes" {  
  bucket = "cic-reportes-${var.entorno}"  
}  
  
variable "entorno" {  
  type = string  
}
```



Terraform CLI

Lee los archivos .tf y habla con las APIs de cada nube a través de proveedores.



Un mismo flujo, muchos proveedores



AWS



Azure



Google Cloud



OpenStack



Kubernetes



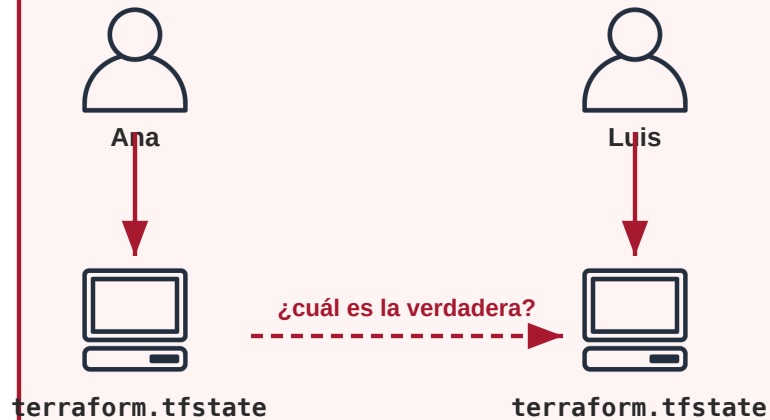
El plan es la red de seguridad

Muestra con símbolos + (crear), ~ (modificar) y - (destruir) qué ocurrirá, antes de tocar nada.

El estado de Terraform

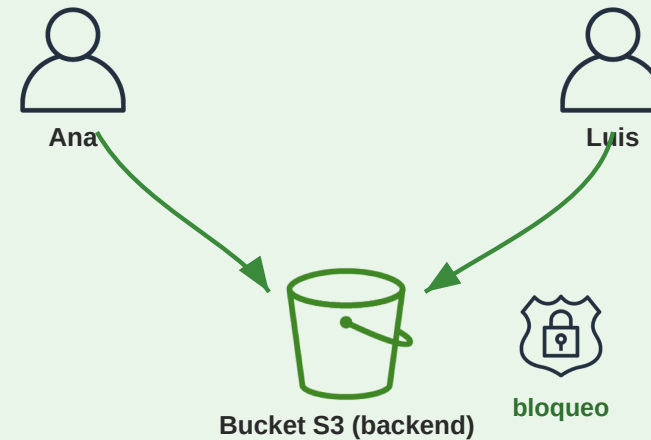
Terraform recuerda lo que creó en un archivo de estado; compartirlo bien es parte del diseño.

Estado local: cada quien con su copia



Dos copias distintas: Terraform duplica o destruye recursos.

Estado remoto: una sola fuente de verdad



Con versionado, cifrado y bloqueo: nadie aplica al mismo tiempo.

¿Qué guarda el estado?

`aws_s3_bucket.reportes`



bucket real: cic-reportes-dev

El mapa entre cada recurso del código y su objeto real. Puede contener datos sensibles: cifra el almacenamiento y limita el acceso.



Bicep: el lenguaje declarativo de Azure

La misma idea, con otra sintaxis: describir recursos y dejar que Azure Resource Manager los cree.

almacenamiento.bicep

```
param entorno string = 'dev'

resource sa 'Microsoft.Storage/storageAccounts@2023-05-01' = {
  name: 'cicrep${entorno}${uniqueString(resourceGroup().id)}'
  location: resourceGroup().location
  sku: { name: 'Standard_LRS' }
  kind: 'StorageV2'
}

output nombre string = sa.name
```

Qué ves en este archivo

parámetro

recurso

salida

Más corto y legible que JSON. El tipo y la versión de API van en la declaración del recurso.

Bicep es un lenguaje de dominio para Azure: es declarativo y sin estado propio.

Compila a una plantilla ARM y la ejecuta Azure Resource Manager



Comandos clave (Azure CLI)

az deployment group what-if

Vista previa de cambios, como el plan

az deployment group create

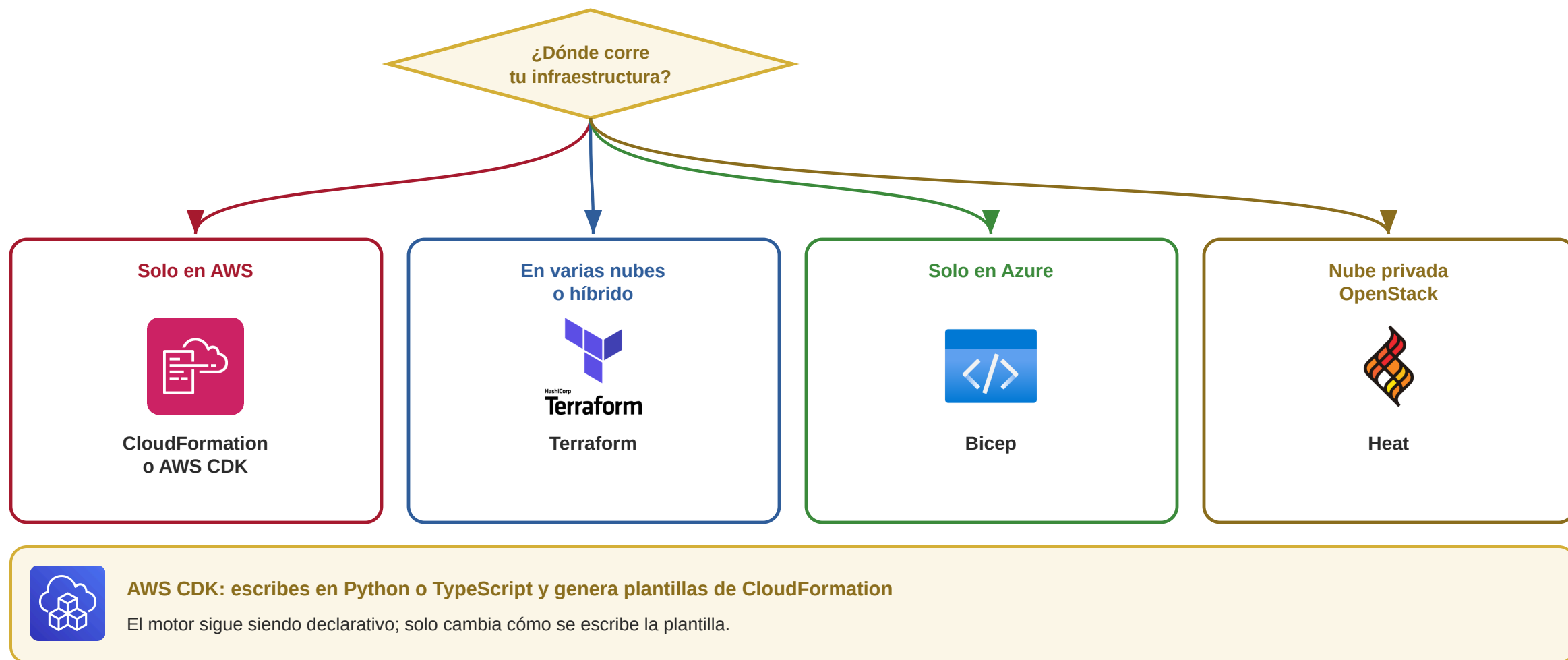
Despliega el archivo .bicep en un grupo de recursos

az bicep build

Compila a JSON de ARM

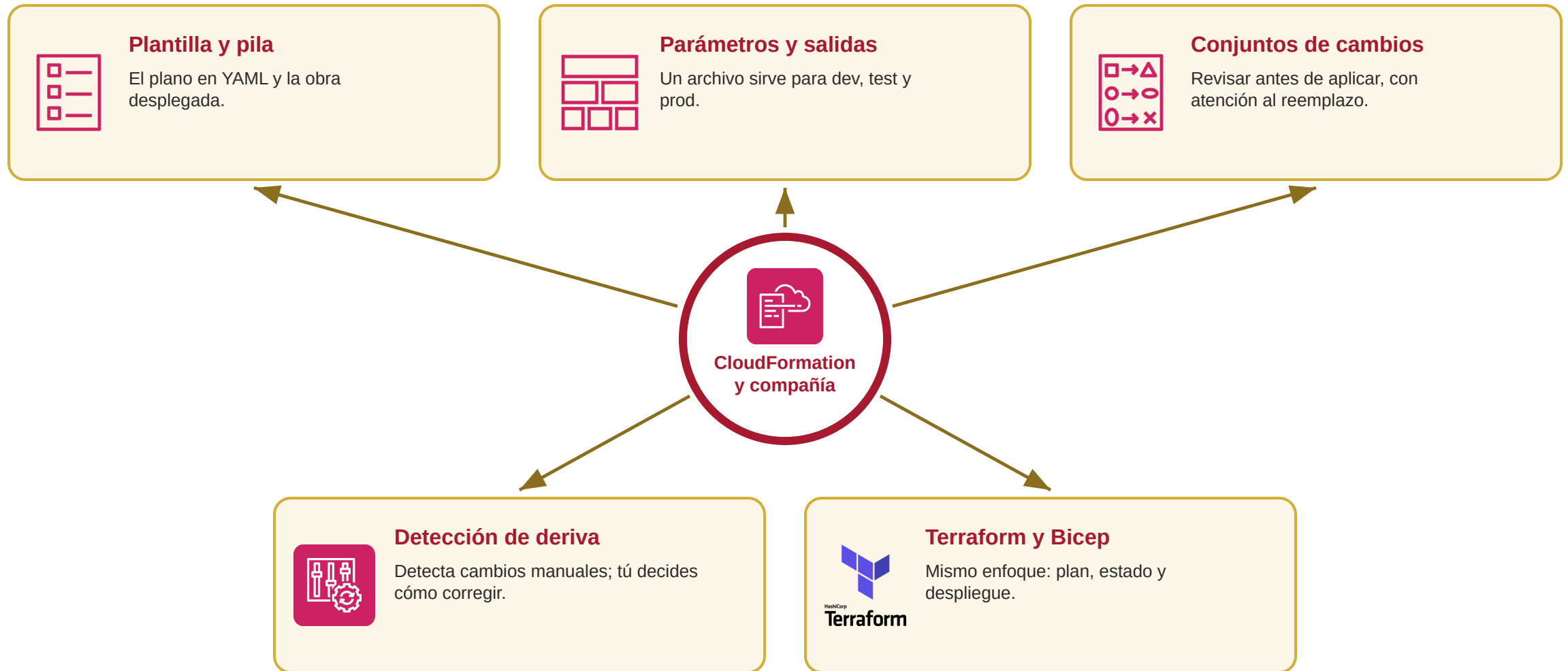
¿Cuál elegir? Depende de dónde corres

Las herramientas comparten la idea; cambian el lenguaje, el alcance y dónde guardan el estado.



Lo que queda de la Parte 2

CloudFormation, Terraform y Bicep aplican el mismo ciclo: declarar, revisar, aplicar y vigilar la deriva.



Entrega continua de la infraestructura

Parte 3 de 4 · Unidad 2

Ing. Rodolfo Cañas Cervantes

Universidad de la Costa (CUC) · Ingeniería de Sistemas

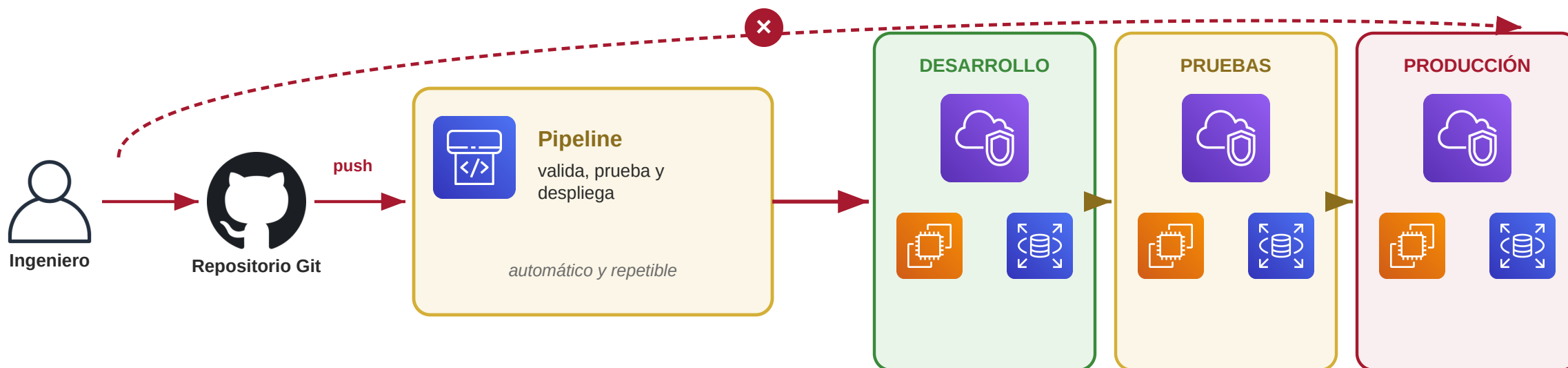
Semana 11 · Periodo 2026-2 · Barranquilla, Colombia

Git, pipelines CI/CD, entornos, pruebas y GitOps para que cada cambio llegue a producción con control.

La plantilla también es software

Si la infraestructura es código, se versiona, se revisa, se prueba y se entrega como cualquier programa.

Acceso directo a producción por consola



Antes

Cada persona entra a producción y cambia lo que cree necesario. No hay historial ni revisión.

Entrega continua

Todo cambio pasa por Git y por el pipeline. La consola queda para mirar, no para cambiar.

Resultado

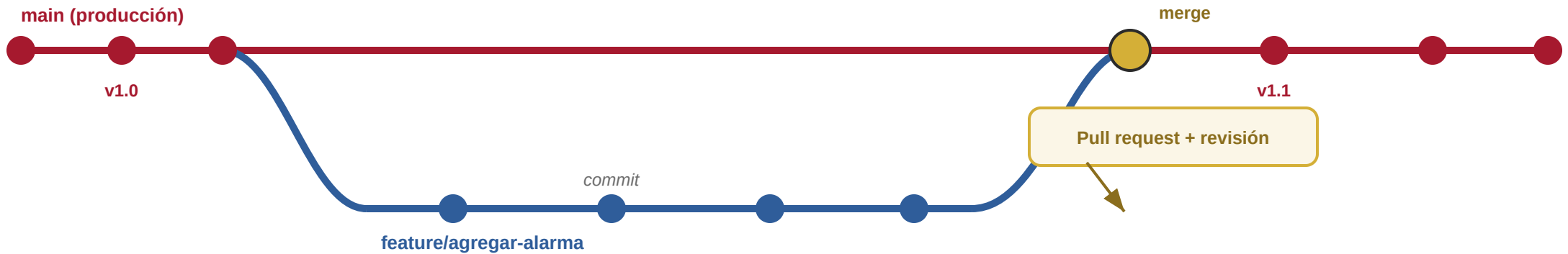
Cambios pequeños, frecuentes, revisados y reversibles.

Control de versiones: la base de todo

Cada cambio de infraestructura es un commit en Git, trabajado en una rama y unido por pull request.



Historial de Git: ramas y pull request



Commit

Punto de restauración con autor, fecha y motivo.



Rama y pull request

Se trabaja aparte y se une solo tras revisión.

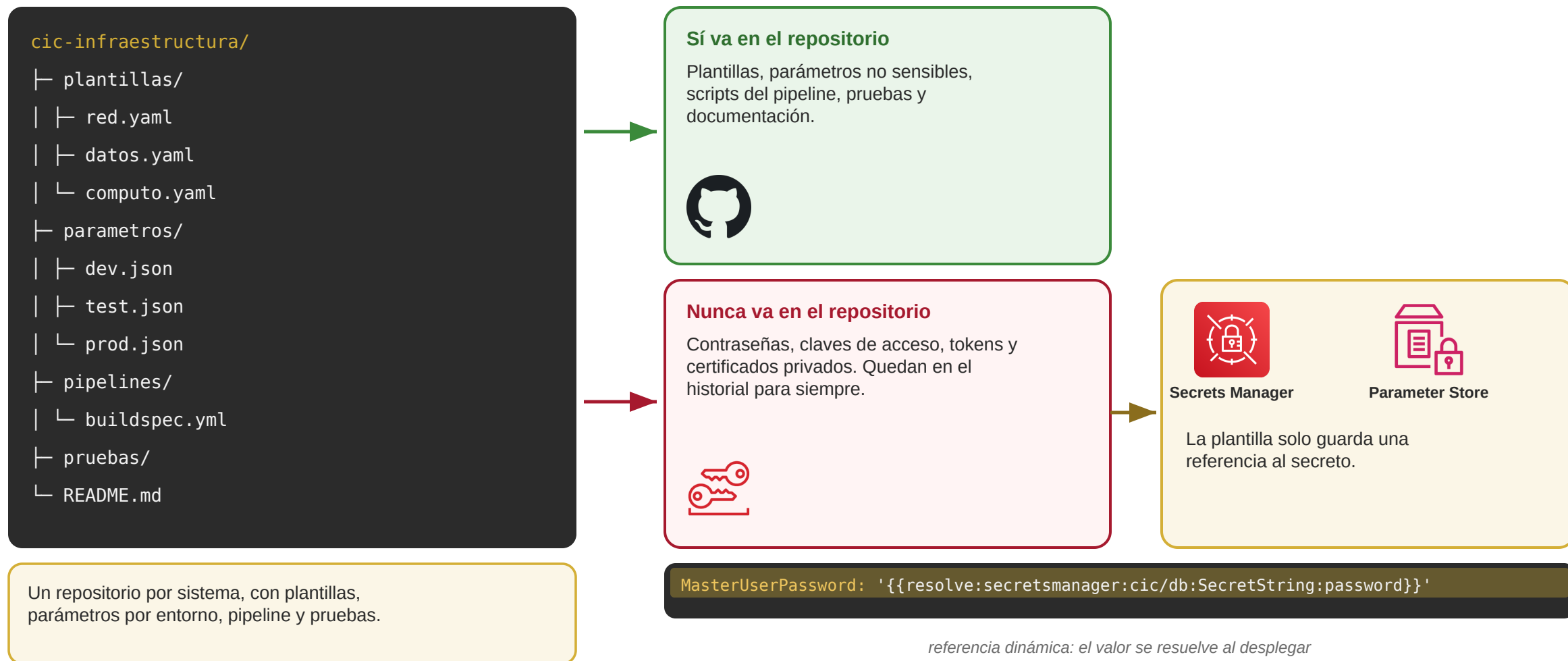


Historial = auditoría

Quién cambió qué, cuándo y por qué, para siempre.

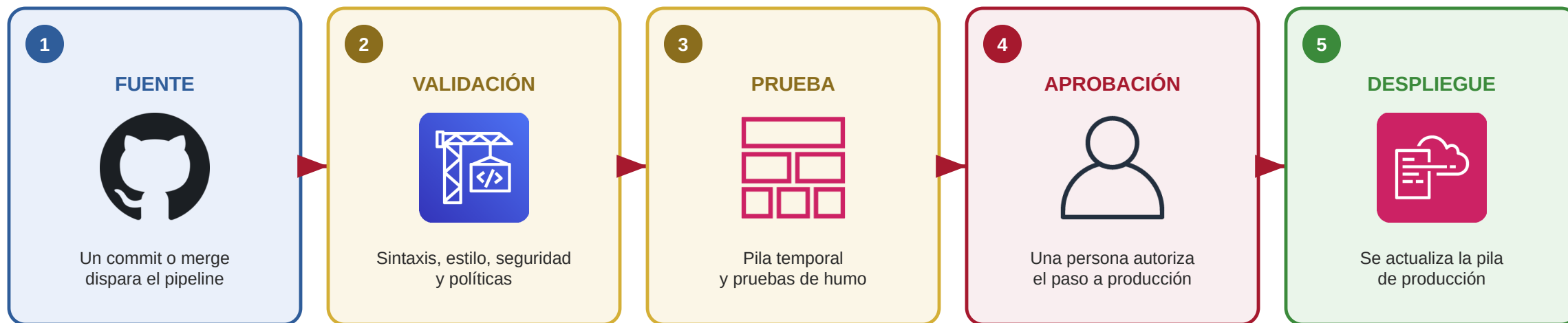
Qué va (y qué no) en el repositorio

El repositorio guarda el plano completo, pero nunca los secretos: esos se referencian, no se copian.



El pipeline: fuente, validación y despliegue

Un pipeline convierte un commit en un despliegue pasando por etapas que fallan rápido y con control.



Integración continua (CI): construir y comprobar cada cambio

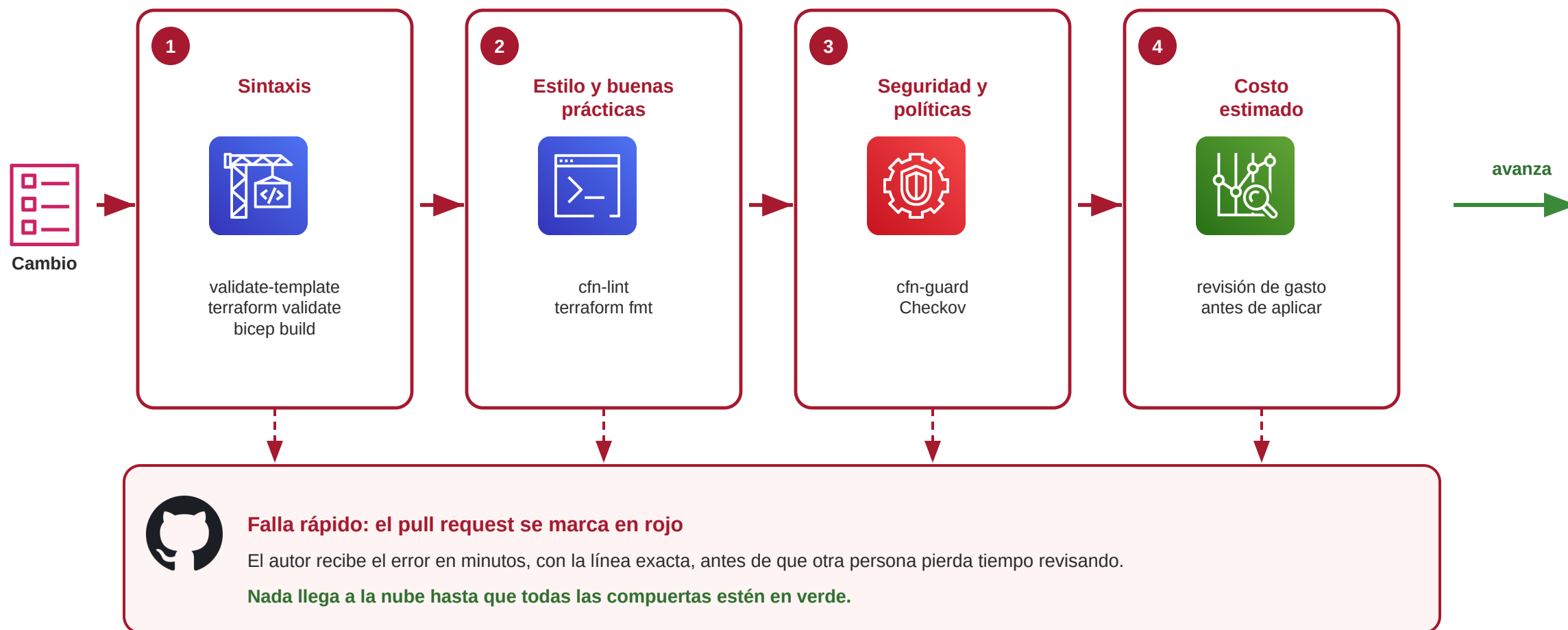
Entrega continua: siempre listo para producción, con aprobación humana

Despliegue continuo: sin aprobación, todo automático hasta producción

La mayoría de las empresas usa entrega continua en producción de infraestructura crítica.

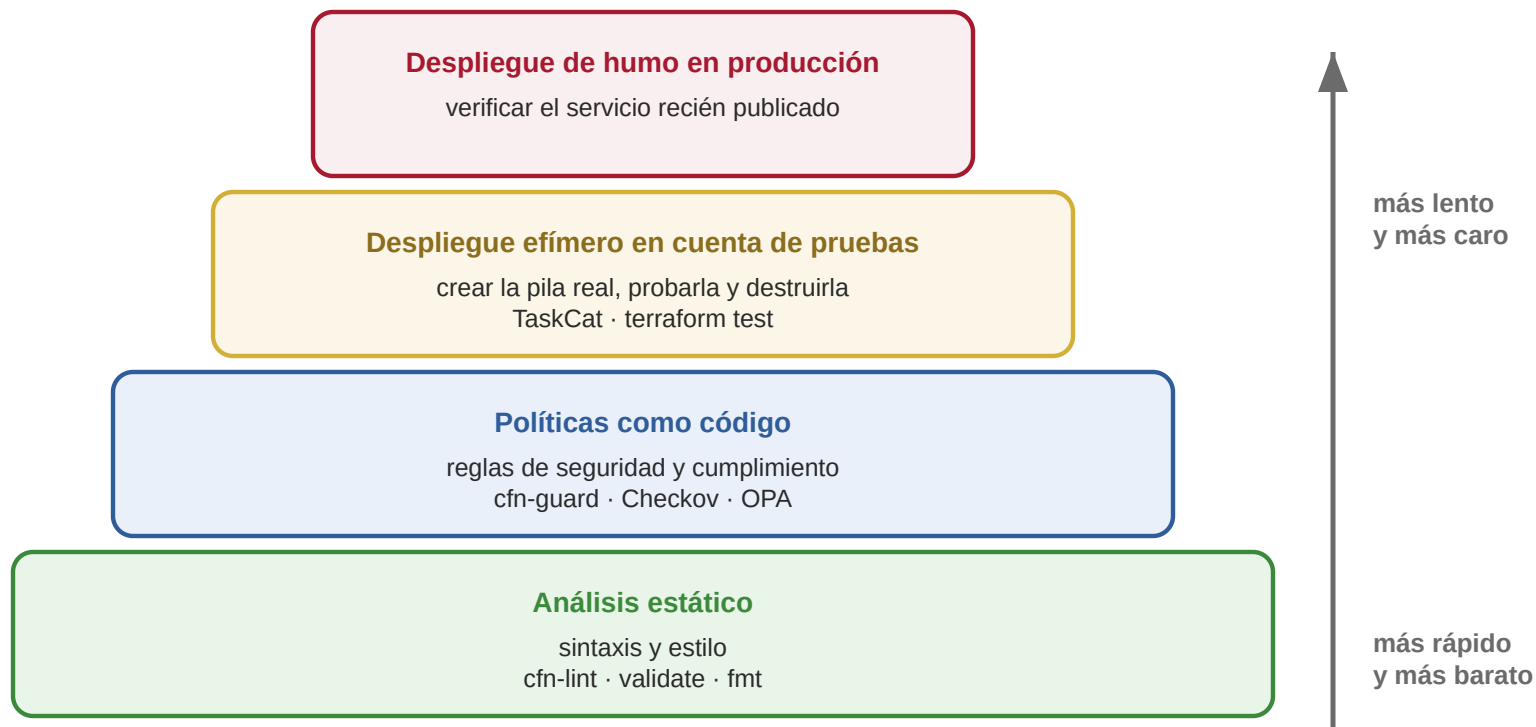
Etapa de validación: cuatro compuertas

Cada cambio atraviesa revisiones automáticas baratas antes de gastar un solo centavo en la nube.



Pruebas de plantillas: la pirámide

Muchas pruebas estáticas rápidas, algunas de políticas y pocas de despliegue real.



Cómo se prueba una plantilla



No se prueba "a ojo": se despliega en una cuenta aislada.



Se comprueba que el servicio responda y que las salidas existan.



Se elimina la pila al final: no queda gasto residual.

Regla práctica: muchas pruebas rápidas en la base y pocas pruebas lentas en la cima.

Entornos: dev, test y prod en cuentas separadas

El mismo pipeline promueve la misma plantilla de un entorno al siguiente, con parámetros distintos.



Cuentas separadas dentro de AWS Organizations: un error en desarrollo no puede tocar producción.

El pipeline asume un rol distinto en cada cuenta; la aprobación manual protege el último paso.

Revisión por pares: el segundo par de ojos

Un cambio de infraestructura se aprueba como un cambio de código: con diff, comentarios y comprobaciones.



Pull request #42: agregar acceso SSH para soporte

infra/plantillas/computo.yaml

SecurityGroupIngress:

- IpProtocol: tcp

FromPort: 22

ToPort: 22

- CidrIp: 0.0.0.0/0

+ CidrIp: 10.20.0.0/16



María (revisora)

Abrir el puerto 22 a todo Internet es riesgoso. Limitémoslo a la red de la oficina.

Autor: corregido en el commit siguiente.

Comprobaciones automáticas



Sintaxis y cfn-lint



Política: sin SSH abierto al mundo



Conjunto de cambios publicado



1 aprobación de un revisor



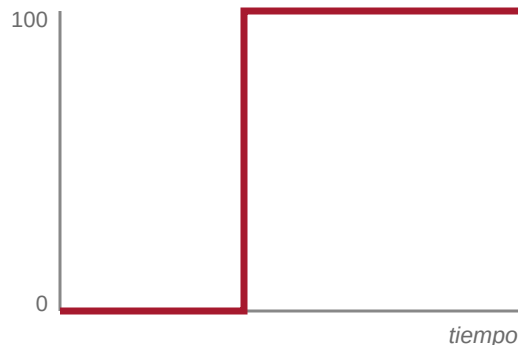
Reglas de la rama main

Protegida: nadie sube directo, mínimo una revisión humana y todas las comprobaciones en verde.

Estrategias de despliegue

Todo de una vez es rápido y arriesgado; gradual, azul-verde y canario reducen el daño si algo falla.

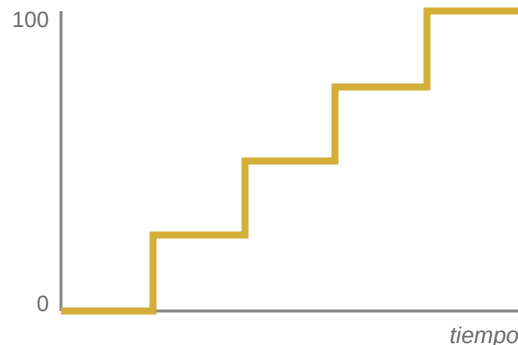
Todo de una vez



Riesgo: alto

Vuelta atrás: volver a desplegar la versión anterior.

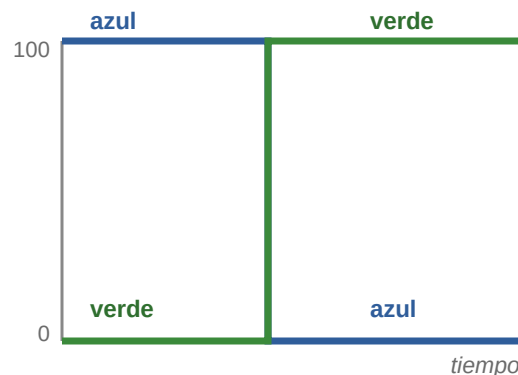
Gradual (rolling)



Riesgo: medio

Se actualiza por lotes; la vuelta atrás es lenta.

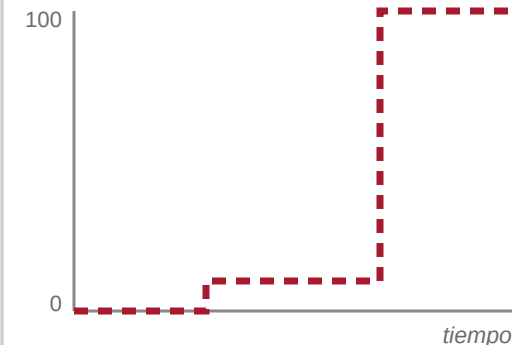
Azul-verde



Riesgo: bajo

Dos entornos completos; volver es cambiar el balanceador.

Canario



Riesgo: bajo

Un 10 % prueba primero; si hay errores, se detiene.



Cuál usar depende del riesgo y del costo

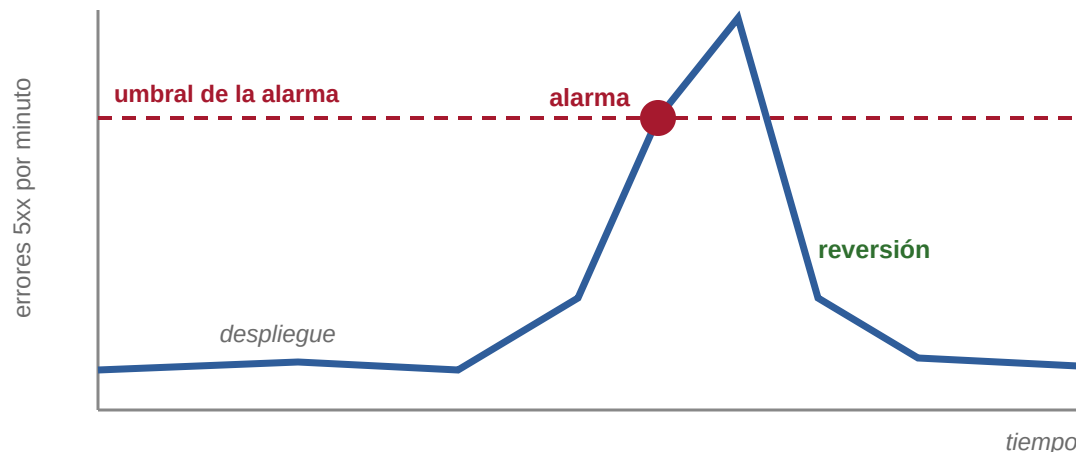
CodeDeploy, ECS y Lambda ofrecen estas estrategias de forma nativa. La infraestructura base (red, datos) se cambia con conjuntos de cambios; las estrategias de arriba se aplican sobre todo a la capa de aplicación.

Más control = más costo temporal

Reversión: volver atrás con confianza

Una alarma puede deshacer un mal despliegue sola; si no, volver a la versión anterior es un commit.

Alarma que dispara la reversión



Reversión automática

1



Alarma de CloudWatch se activa

2



CloudFormation detecta la alarma y cancela la actualización

3



Los recursos vuelven a la versión anterior

Se configura con alarmas de rollback en la pila o con reversión automática en CodeDeploy.

Sin intervención humana a las 3 de la mañana.

Reversión manual por Git: volver es un commit más



git revert del cambio malo



El pipeline corre de nuevo



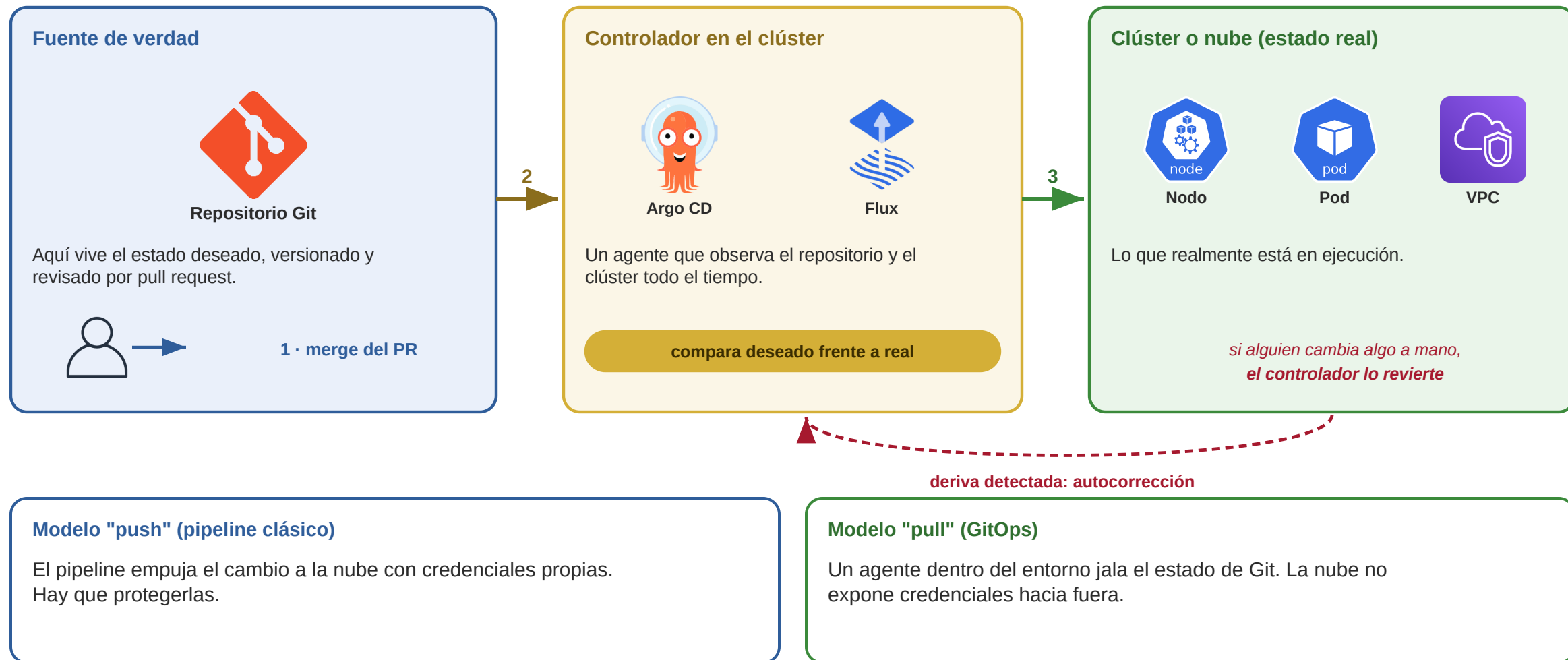
Se despliega la versión anterior



Queda registrado en el historial

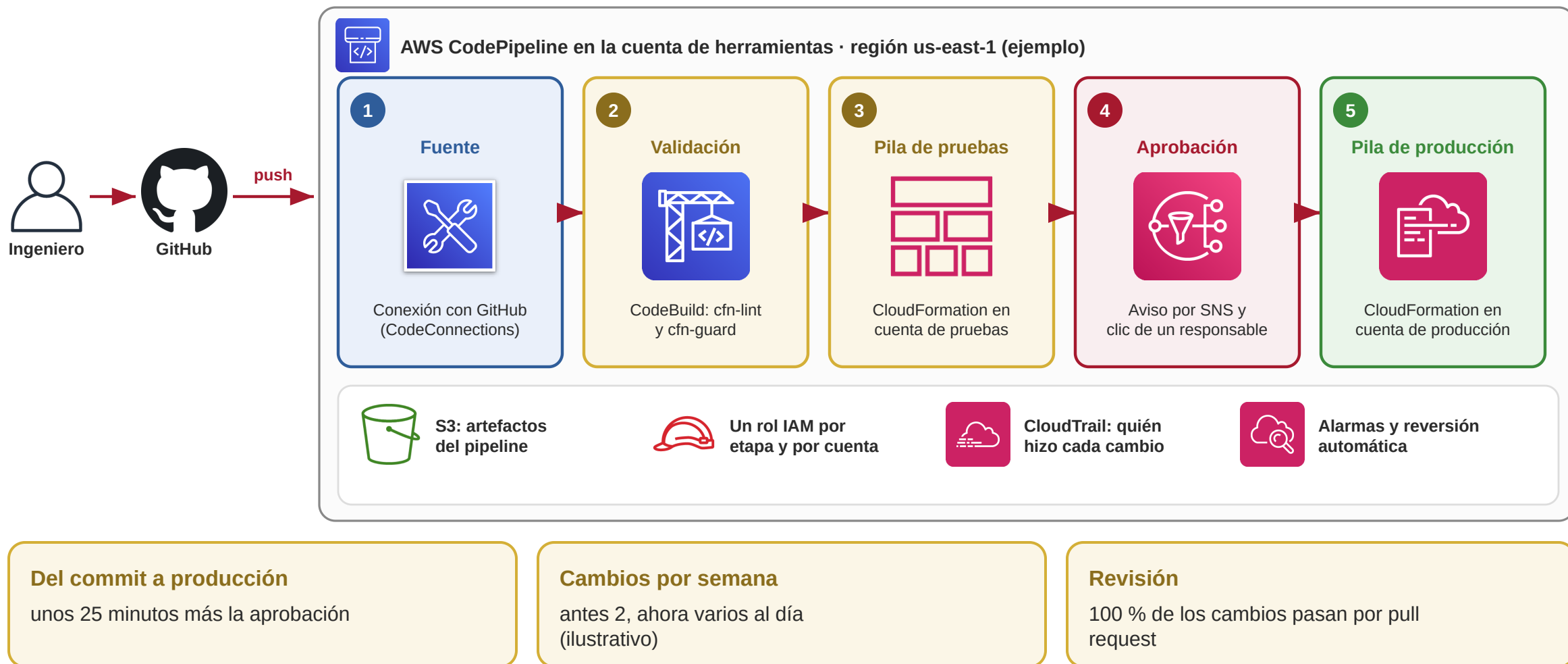
GitOps: Git como fuente de verdad

Un controlador compara continuamente Git con la realidad y corrige cualquier diferencia.



Caso CIC: el pipeline completo en AWS

Del push en GitHub a la pila de producción: validación, pruebas, aprobación y despliegue sin consola.



Seguridad de la cadena de entrega

El pipeline también es un blanco: se protege con credenciales temporales y roles de mínimo privilegio.

Antes: claves de acceso largas



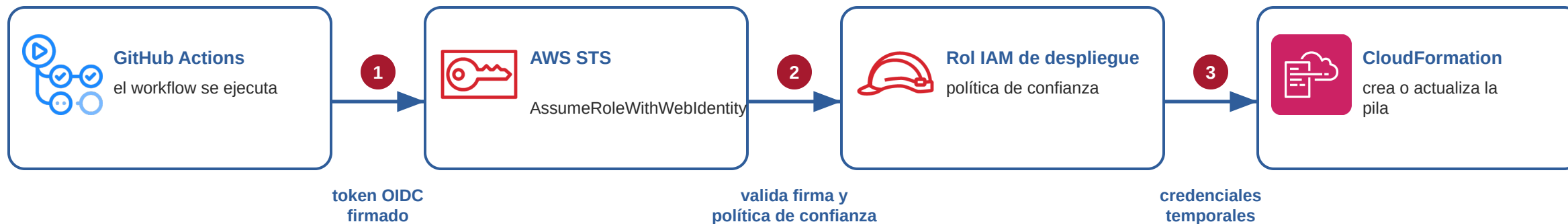
guardadas como secreto
del repositorio

Se filtran, no caducan, sirven desde cualquier lugar.

Ahora: federación OIDC

Sin claves guardadas. Cada ejecución recibe credenciales temporales de pocos minutos.

Ligadas a un repositorio y a una rama concretos.



política de confianza del rol (extracto)

```
"Condition": {
  "StringEquals": {
    "token.actions.githubusercontent.com:aud": "sts.amazonaws.com",
    "token.actions.githubusercontent.com:sub": "repo:cic/infra:ref:refs/heads/main"
  }
}
```

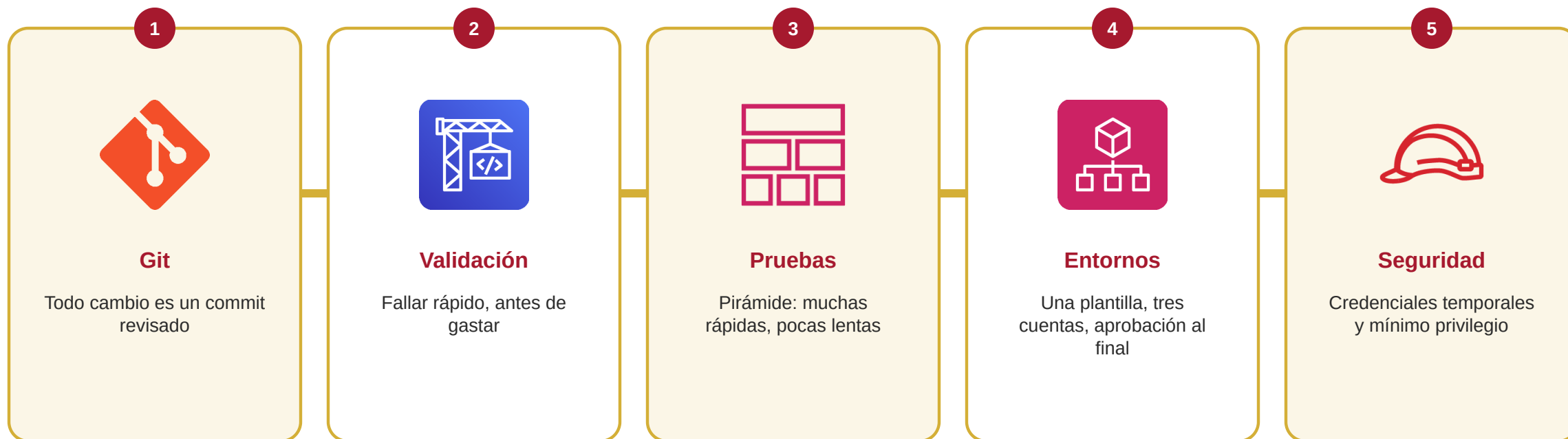


Mínimo privilegio

El rol solo puede desplegar las pilas de este sistema, no administrar la cuenta.

Lo que queda de la Parte 3

Del commit a producción por un camino repetible: Git, validación, pruebas, entornos y seguridad.



Lo que queda de la Parte 3

Un cambio de infraestructura llega a producción por un camino repetible: Git, validación, pruebas, aprobación.

Si algo falla, se vuelve atrás con una alarma o con un commit; si alguien cambia algo a mano, GitOps lo corrige.

Repaso de la Unidad 2 y laboratorio de IaC

Parte 4 de 4 · Unidad 2

Ing. Rodolfo Cañas Cervantes

Universidad de la Costa (CUC) · Ingeniería de Sistemas

Semana 11 · Periodo 2026-2 · Barranquilla, Colombia

Cómputo, bases de datos, identidad y alta disponibilidad integrados en una arquitectura desplegada con código.

La Unidad 2 en un mapa

Cinco semanas, cinco preguntas de ingeniería; la última explica cómo construir las otras cuatro con código.



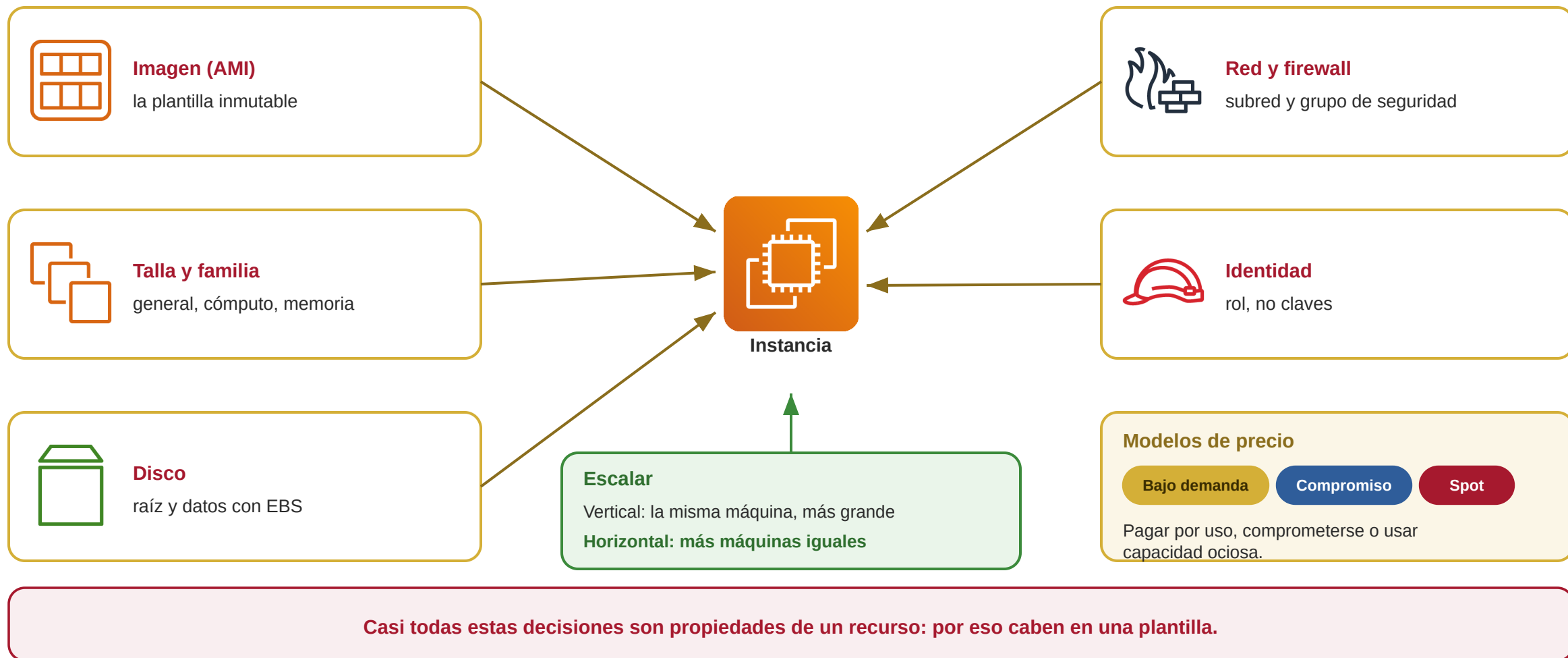
Cinco capas de una misma arquitectura

Cómputo, datos, acceso y resiliencia son las piezas; la automatización es el hilo que las construye igual cada vez.

La semana 11 no agrega una pieza nueva: cambia la forma de armar todas las anteriores.

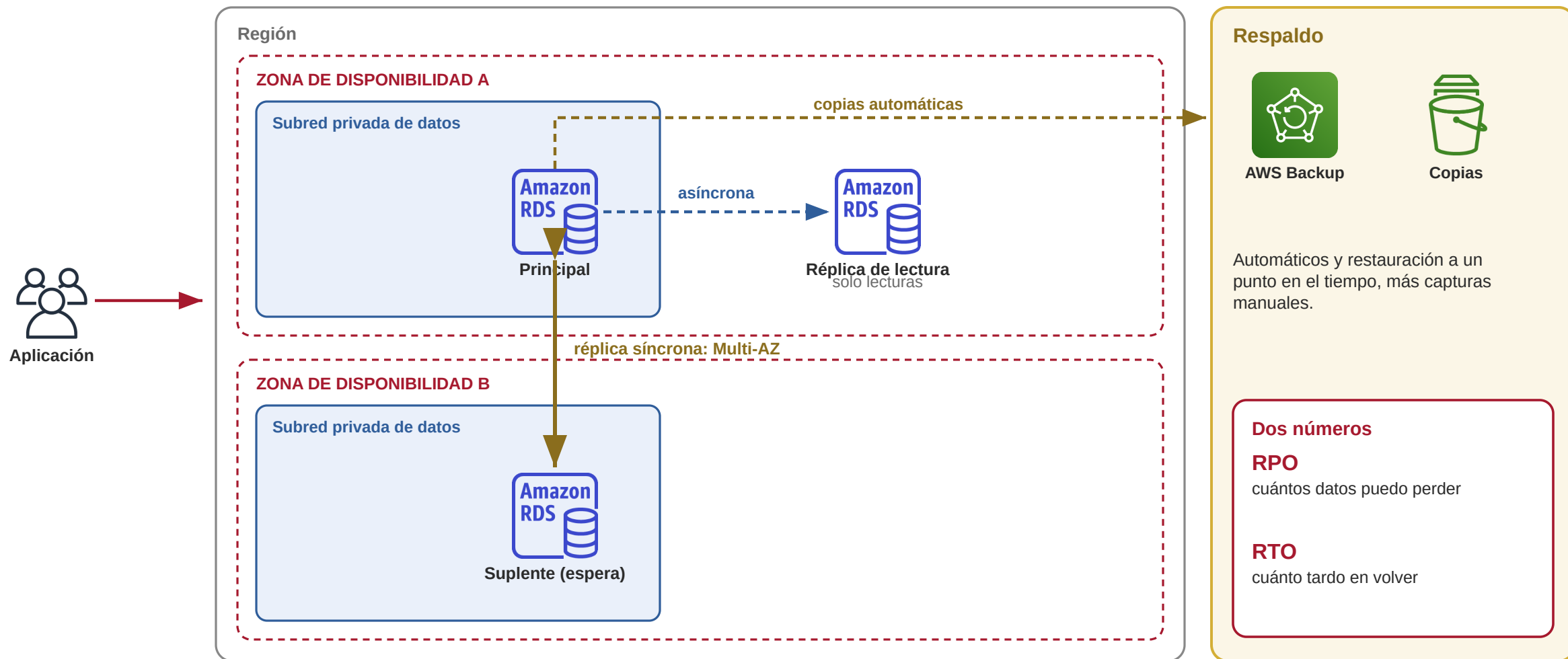
Semana 7: la instancia y sus decisiones

Una instancia es una imagen, una talla, un disco, una red y una identidad; todo lo demás es combinación.



Semana 8: datos disponibles y respaldados

Multi-AZ protege de una falla de zona; los respaldos protegen de errores humanos y corrupción.



Semana 9: identidad y mínimo privilegio

Los permisos efectivos son la intersección de varias capas; las credenciales deben ser temporales.

Capas que limitan los permisos

SCP de la organización (techo para toda la cuenta)

Límite de permisos del rol (permission boundary)

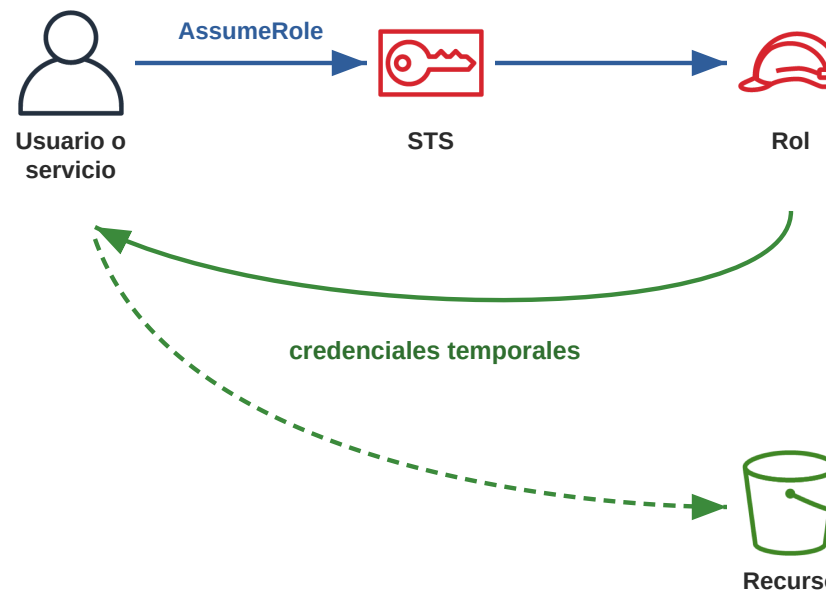
Política de identidad del rol



Permisos efectivos: solo lo que las tres capas permiten a la vez.

Un Deny explícito siempre gana sobre cualquier Allow.

Rol: credenciales temporales en lugar de claves



También en la semana 9

Federación y SSO

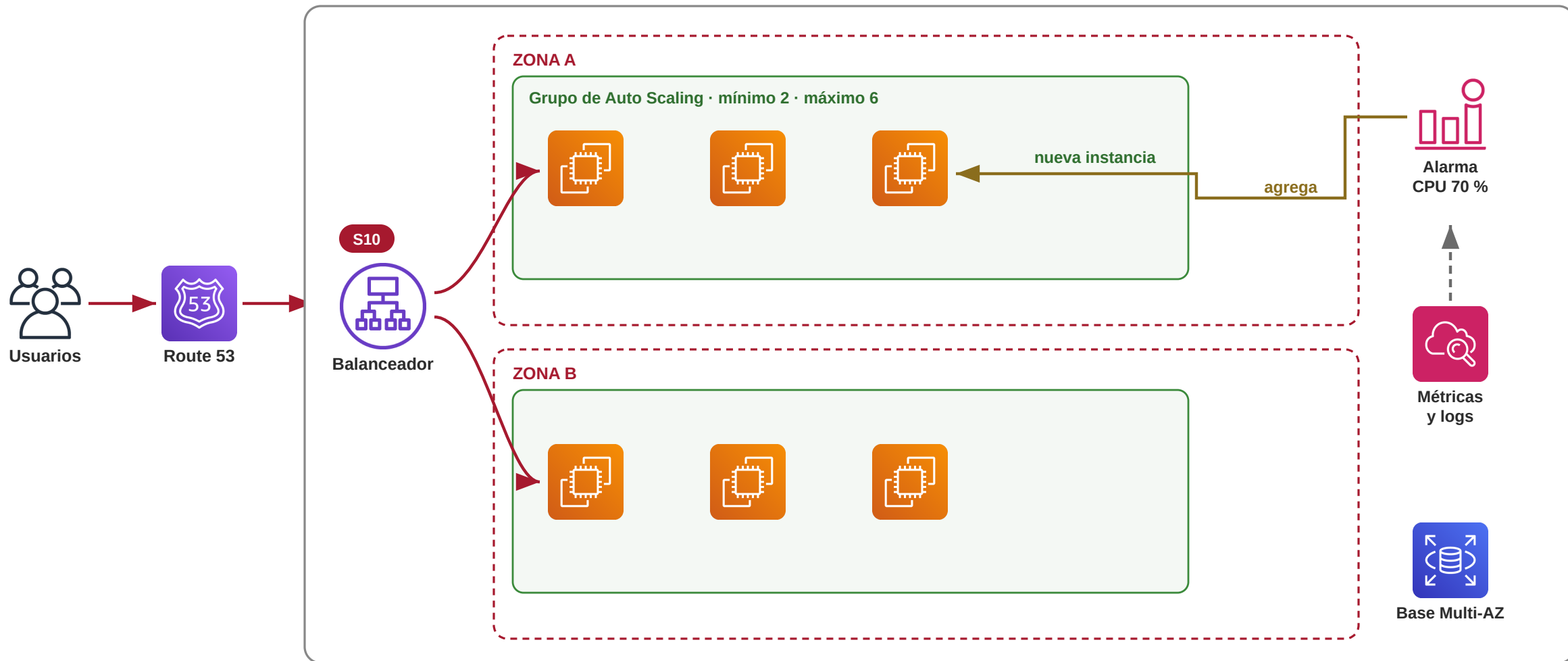
Cognito

Multi-cuenta

Gobernanza y costos

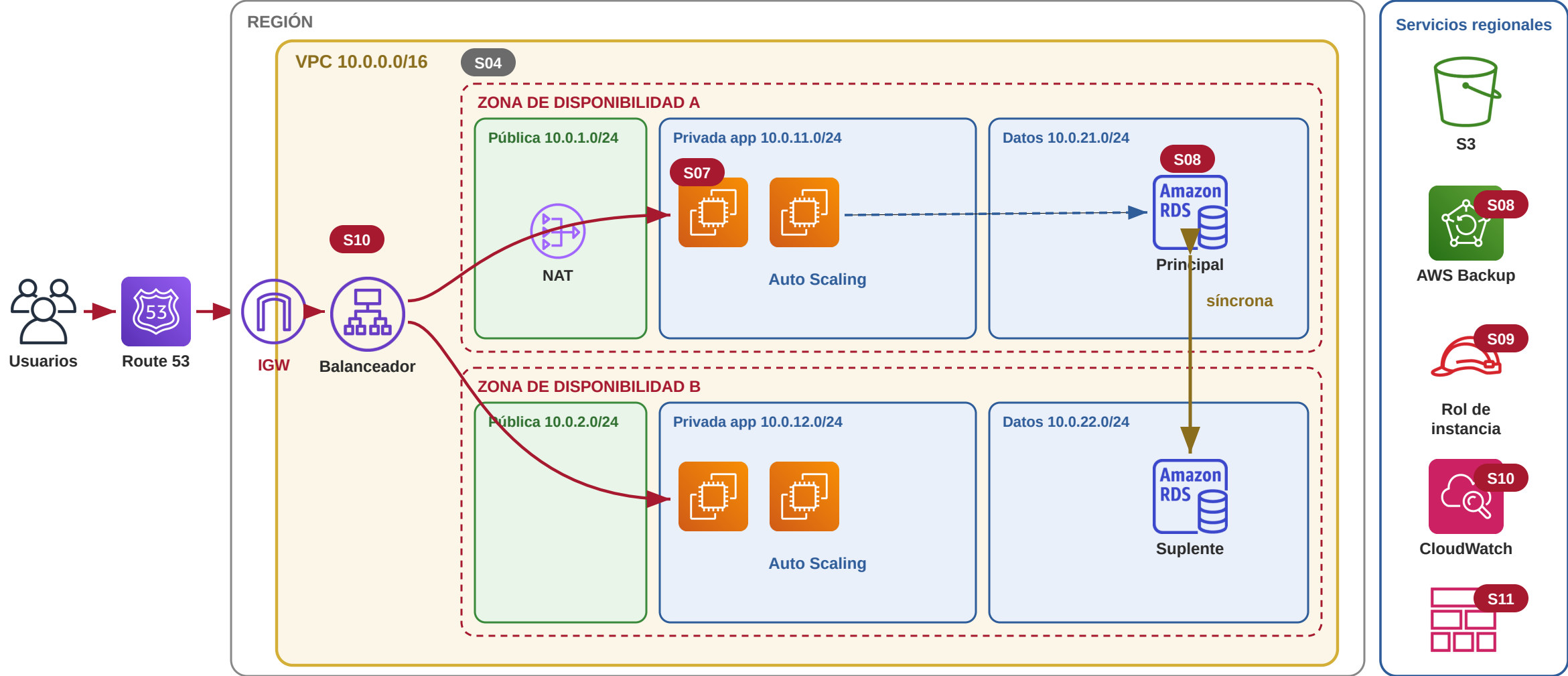
Semana 10: resistir y crecer solos

El balanceador reparte, Auto Scaling ajusta la cantidad y las alarmas deciden cuándo.



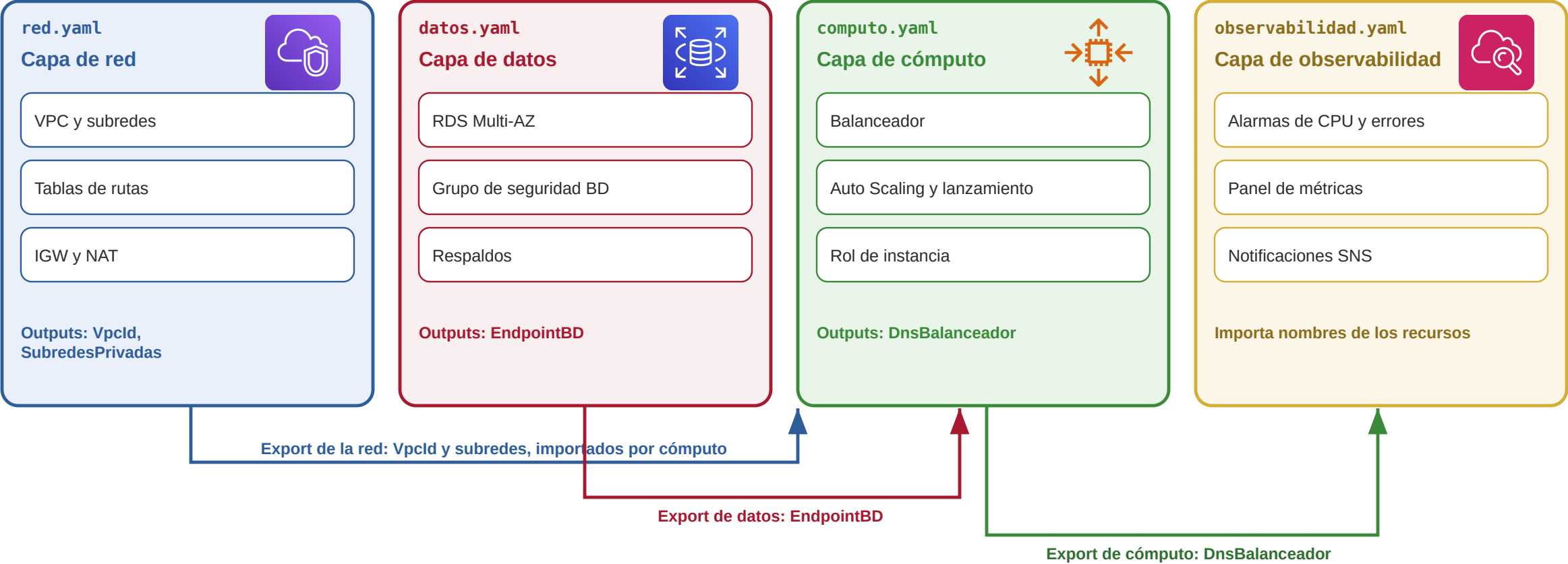
Arquitectura de referencia de la CIC

Cómputo, datos, identidad y resiliencia en una sola arquitectura: lo que desplegaremos con una plantilla.



La arquitectura como pilas en capas

Dividir por frecuencia de cambio permite desplegar y revertir cada capa sin tocar las demás.



Cada capa es una pila con su propio ciclo de vida: la red casi no cambia; el cómputo cambia todos los días.

Alternativa: pilas anidadas (nested stacks) desde una plantilla raíz.

Laboratorio de laC: qué se despliega

Una plantilla corta crea red mínima, servidor, bucket y salidas; luego se cambia y se vigila.

Qué vas a desplegar en el laboratorio

VPC de laboratorio

Subred pública



Servidor web



Grupo de seguridad

Parámetro: Entorno



Bucket



Todo desde una sola plantilla YAML, creada como una pila.

Salidas de la pila: dirección pública del servidor y nombre del bucket.

Parámetro: tamaño

Etiquetas

Salidas útiles

Cuatro fases

1

Construir

Escribir y validar la plantilla

2

Desplegar

Crear la pila y verificar

3

Cambiar

Conjunto de cambios con otro tamaño

4

Vigilar

Provocar y detectar deriva; eliminar

Laboratorio paso a paso

Seis comandos recorren todo el ciclo: validar, crear, consultar, cambiar, vigilar y limpiar.

1 Validar

```
aws cloudformation validate-template \  
--template-body file://lab.yaml
```

2 Crear la pila

```
aws cloudformation create-stack \  
--stack-name lab-iac \  
--template-body file://lab.yaml \  
--parameters ParameterKey=Entorno,ParameterValue=dev
```

3 Ver salidas

```
aws cloudformation describe-stacks \  
--stack-name lab-iac
```

4 Cambiar

```
aws cloudformation create-change-set ...  
aws cloudformation execute-change-set ...
```

5 Detectar deriva

```
aws cloudformation detect-stack-drift \  
--stack-name lab-iac
```

6 Eliminar

```
aws cloudformation delete-stack \  
--stack-name lab-iac
```

Entregables del laboratorio

Cinco evidencias cuentan la historia completa: construir, desplegar, cambiar, vigilar y limpiar.

- 1



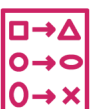
Plantilla YAML comentada
Con parámetros, recursos y salidas


- 2



Pila en CREATE_COMPLETE
Captura de la consola con las salidas


- 3



Conjunto de cambios
Vista previa y decisión sobre el reemplazo


- 4



Informe de deriva
Recurso modificado y valor esperado frente al real


- 5



Pila eliminada
Evidencia de limpieza: sin recursos residuales





Evaluación
10 % de la nota del curso

Los criterios y la rúbrica detallada los publica el docente junto con la guía del laboratorio.

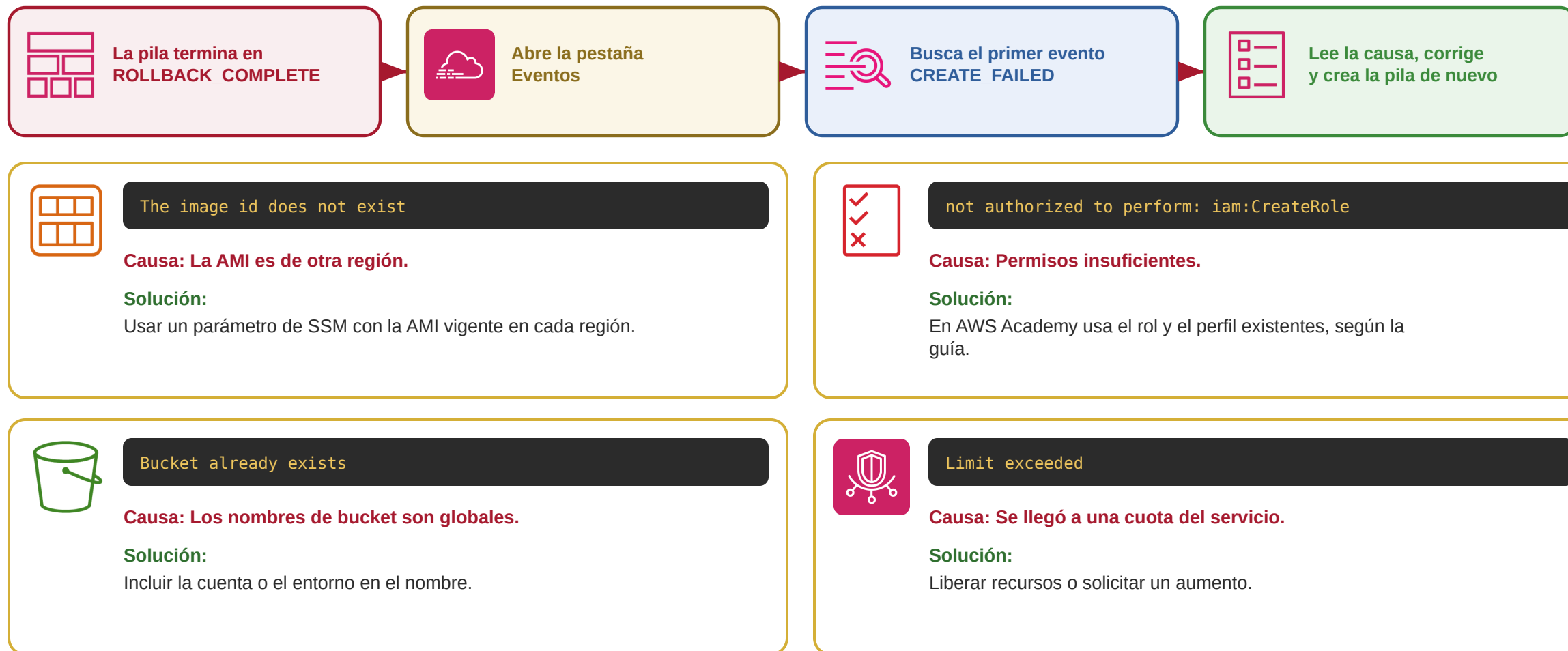
Buenas prácticas esperadas:

- Que la plantilla sea válida y legible
- Que el despliegue sea reproducible
- Que expliques cada decisión
- Que no queden recursos activos

Antes de enviar: revisa que tu captura muestre el nombre de la pila y su estado.

Cuando la pila falla: cómo diagnosticar

El mensaje de error está en el primer evento fallido; casi siempre apunta a una de cuatro causas.



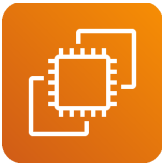
Orientación para el Parcial 2

Cinco semanas, tres habilidades: elegir, justificar y conectar las piezas de la arquitectura.



Parcial 2: 20 % de la nota, evaluado con rúbrica

S07 · Cómputo



Elegir talla y familia

Comparar modelos de precio

Vertical frente a horizontal

S08 · Datos



Multi-AZ frente a réplica

Respaldo, retención y PITR

Calcular RTO y RPO

S09 · Identidad

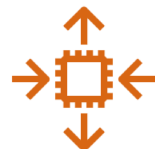


Roles y credenciales temporales

Evaluación de políticas

Mínimo privilegio

S10 · Resiliencia



Balanceador y Auto Scaling

Alarmas y métricas

Diseño multi-AZ

S11 · Automatización



Declarativo e idempotente

Pila, cambios y deriva

Pipeline y entornos

Cómo estudiar

1

Dibuja de memoria la arquitectura de referencia

2

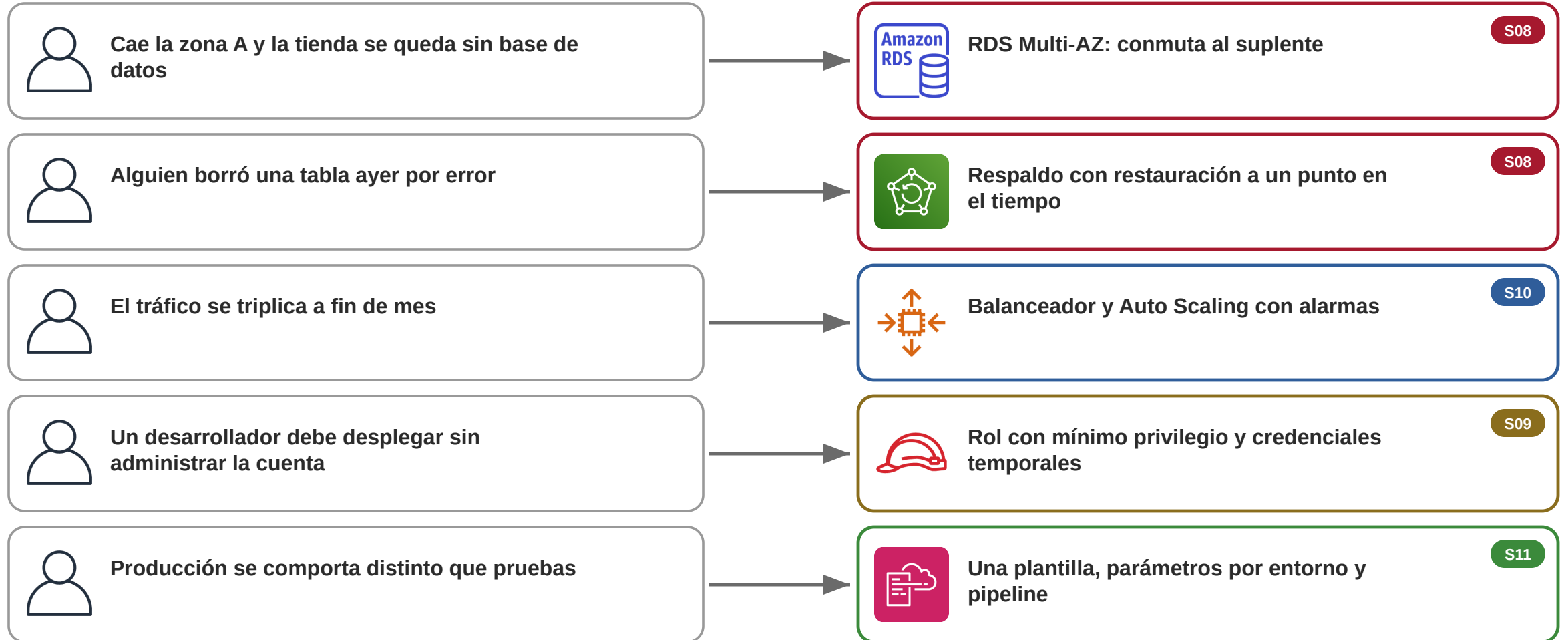
Explica cada decisión: por qué esta pieza y no otra

3

Resuelve los escenarios de la siguiente diapositiva

Escenarios tipo: del problema a la pieza

Cada situación apunta a una decisión de la unidad; practica enlazarlas rápido.



Semana 11 en cuatro ideas y adelanto

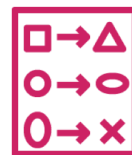
La infraestructura como código se resume en declarar, revisar, entregar y vigilar.

1 Declarar



El estado deseado va en un archivo; el motor calcula los pasos.

2 Revisar



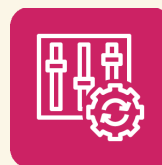
Conjuntos de cambios y pull requests antes de aplicar.

3 Entregar



Un pipeline lleva el cambio por dev, test y prod.

4 Vigilar



Deriva detectada y corregida; reversión disponible.

Adelanto: Semana 12

Caché y distribución de contenido en la nube



CloudFront



ElastiCache



Route 53

CloudFront: red de borde

ElastiCache: caché en memoria

Route 53: políticas de enrutamiento

La arquitectura de hoy se acelera acercando los datos al usuario.

Trae tu arquitectura de referencia